
UNIT 3 UNINFORMED & INFORMED SEARCH

Structure	Page No
3.0 Introduction	92
3.1 Objectives	94
3.2 Formulating search in state space	95
3.2.1 Evaluation of the Search Algorithm	97
3.3 Uninformed Search	97
3.3.1 Breath-First search (BFS)	98
3.3.2 Time and space complexity of BFS	101
3.3.3 Advantages & disadvantages of BFS	102
3.3.4 Depth First Search (DFS)	104
3.3.5 Performance of DFS algorithm	104
3.3.6 Advantages and disadvantages of DFS	104
3.3.7 Comparison of BFS and DFS	105
3.4 Iterative Deepening Depth First search (IDDFS)	106
3.4.1 Time and space complexity of IDDFS	108
3.4.2 Advantages and Disadvantages of IDDFS	108
3.5 Bidirectional search	109
3.6 Comparison of Uninformed search strategies	111
3.7 Informed (heuristic) search	112
3.7.1 Strategies for providing heuristics information	113
3.7.2 Formulation of Informed (heuristic) search problem as a state space	114
3.7.3 Best-First search	115
3.7.4 Greedy Best first search	115
3.8 A* Algorithm	120
3.8.1 Working of A* algorithm	122
3.8.2 Advantages and disadvantages of A* algorithm	124
3.8.3 Admissibility properties of A* algorithm	124
3.8.4 Properties of heuristic algorithm	125
3.8.5 Results on A* algorithm	126
3.9 Problem reduction search	126
3.9.1 Problem definition in AND-OR graph	127
3.9.2 AO* algorithm	130
3.9.3 Advantages of AO* algorithm	131
3.10 Memory Bound heuristic search	131
3.10.1 Iterative Deepening A* (IDA*)	131
3.10.2 Working of IDA*	132
3.10.3 Analysis of IDA*	133
3.10.4 Comparison of A* and IDA* algorithm	134
3.11 Recursive Best First search (RBFS)	134
3.11.1 Advantages and disadvantages of RBFS	135
3.12 Summary	137
3.13 Solutions/Answers	140
3.14 Further Readings	144

3.0 INTRODUCTION

Before an AI problem can be solved, it must be represented as a **state space**. In AI, a wide range of problems can be formulated as a search problem. The process of searching means a sequence of action that takes you from an initial state to a goal state, as shown in the following figure 1.



Fig 1: A sequence of actions in a search space from the initial to the goal state.

In Unit 2, we have already examined the concept of a **state space** and adversarial (game playing) search strategy. In many applications, there might be multiple agents or persons searching for solutions in the same solution space. In adversarial search, we need a path to take action towards the winning direction, and to find a path, we need different types of search algorithms.

Search algorithms are one of the most important areas of Artificial Intelligence. This unit will explain all about the search algorithms in AI, which explore the search space to find a solution.

In Artificial Intelligence, Search techniques are **universal problem-solving methods**. Rational agents or **Problem-solving agents** in AI mostly use these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are goal-based agents and use atomic representation.

One **disadvantage** of state space representation is that it is not possible to visualize all states for a given problem. Also, the resources of the computer system are limited to handle a huge state space representation. However, many problems in AI take the form of state-space search.

- The states may represent legal board configurations in a game, locations such as towns and cities in a route-finding problem, or collections of mathematical propositions.
- A state-space is the configuration of the possible states and how they connect to each other, e.g., the legal moves between states.
- When we don't have an algorithm that tells us definitively how to negotiate the state-space, we need to search the state-space to find an optimal path from a start state to a goal state,

We can only decide what to do (or where to go) by considering the possible moves from the current state and trying to look ahead as far as possible. Chess, for example, is a very difficult state space search problem.

Searching is the process of looking for the solution to a problem through a set of possibilities (state space). In general, the searching process starts from the initial state (root node) and proceeds by performing the following steps:

- Check whether the current state is the goal state or not?
- Expand the current state to generate the new sets of states.
- Choose one of the new states generated for search depending upon the search strategy (for example, BFS, DFS, etc.).
- Repeat steps 1 to 3 until the goal state is reached or there are no more state tubes expanded.

Evaluation (properties) of search strategies: A search strategy is characterized by the sequence in which nodes are expanded. Any search algorithm is commonly evaluated according to the following four criteria. The following four essential properties of search algorithms are used to compare the efficiency of any search algorithm.

Completeness: A search algorithm is said to be complete if it guarantees returning a solution if one exists.

Optimality/Admissibility: If the solution found by an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution.

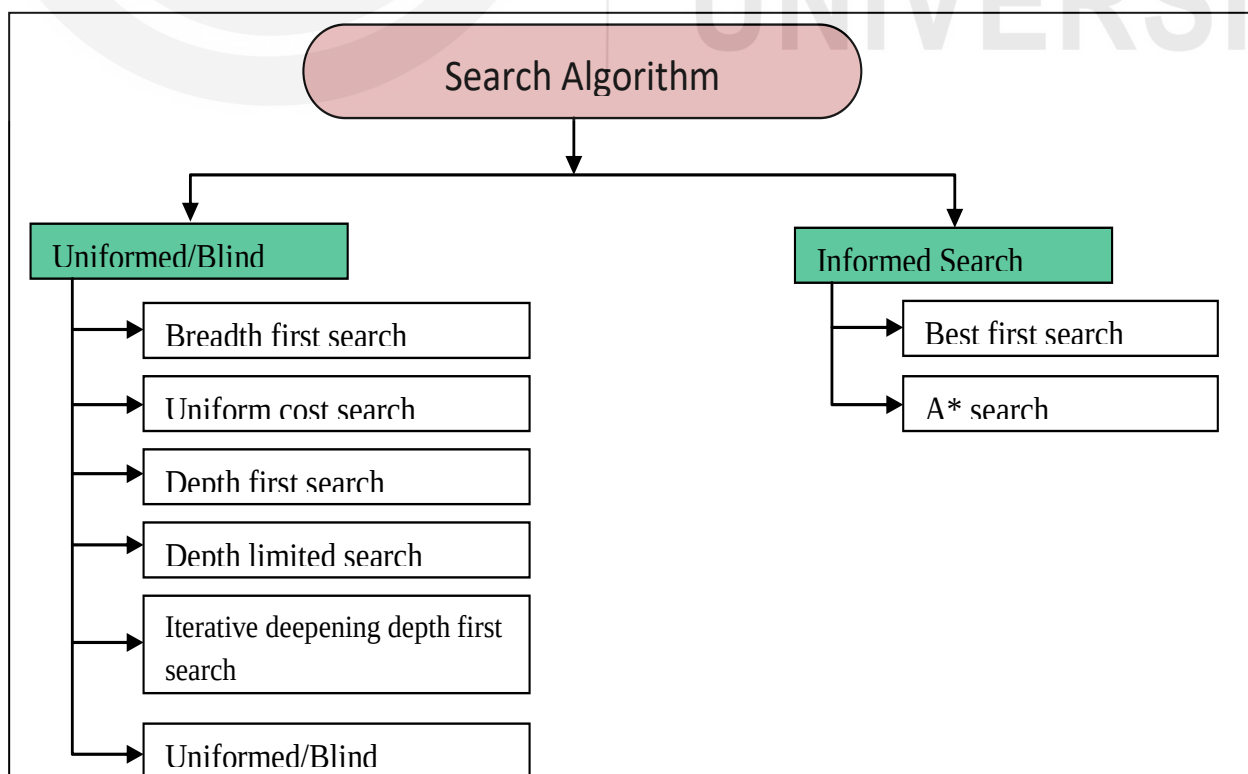
Time Complexity: Time complexity is a measure of the time required for an algorithm to complete its task. Usually measured in terms of the number of nodes expanded during the search.

Space Complexity: It is the maximum storage space required at any point during the search. Usually measured in terms of the maximum number of nodes in memory at a time.

Time and space complexity are measured in terms of:

- b - max branching factor of the search tree
- d - depth of the least-cost solution
- m - max depth of the search tree (may be infinity)

In all search algorithms, the order in which nodes are expanded distinguishes them from one another. There are two broad classes of search methods:



Uninformed search is also called brute force search, Blind search, or Exhaustive search. It is called blind search because of the way in which the search tree is searched without using any information about the search space. It is called brute force because it assumes no additional knowledge other than how to traverse the search tree and how to identify the leaf nodes and goal nodes. This search ultimately examines every node in the tree until it finds a goal state.

Informed search is also called a Heuristic (or guided) search. These are the search techniques where additional information about the problem is provided in order to guide the search in a specific direction. A heuristic is a method that might not always find the best solution, but is guaranteed to find a good solution in a reasonable time. By sacrificing completeness, it increases efficiency.

The following table summarizes the differences between uninformed and informed search:

Uninformed Search	Informed Search
No information about the path cost from the current state to the goal state. It doesn't use domain-specific knowledge during the search process.	The path cost from the current state to the goal state is calculated to select the minimum cost path as the next state. It uses domain-specific knowledge for the searching process.
It finds solutions more slowly than informed search	It finds solutions more quickly.
Less efficient	More efficient
Cost is high.	Cost is low
No guidance is provided regarding the solution; the problem must be solved using only the given information.	It provides guidance toward finding a solution. Additional information can be added as an assumption to solve the problem.
Examples are: ▪ Depth First Search, ▪ Breadth First Search, ▪ Depth limited search, ▪ Iterative Deepening DFS, ▪ Bi-directional search	Examples are: ▪ Best first search ▪ Greedy search ▪ A* search

3.1 OBJECTIVES

After studying this unit, you should be able to:

- Differentiate the Uninformed and informed search algorithms
- Formulate the search problem in the form of a state space
- Explain the differences between various uninformed search approaches, such as BFS, DFS, IDDFS, and Bi-directional search.
- Evaluate the various Uninformed search algorithms with respect to Time, space, and Optimality/Admissibility criteria.

- Explain Informed search, such as Best-First search and A* algorithm.
- Differentiate between the advantages and disadvantages of heuristic search: A* and AO* algorithms
- Differentiate between memory-bound search: Iterative Deepening A* and Recursive Best-First Search.

3.2 FORMULATING SEARCH IN STATE SPACE

A state space is a graph (V, E) where V is a set of nodes and E is a set of arcs, where each arc is directed from one node to another node.

- **V: a node is a data structure that contains** state description, plus, optionally, other information related to the parent of the node, an operation to generate the node from that parent, and other bookkeeping data.
- **E: Each arc** corresponds to an applicable action/operation. The source and destination nodes are called parent (**immediate predecessor**) and child (**immediate successor**) nodes with respect to each other. Ancestors (also called predecessors) and descendants (also called successors) node. Each arc has a fixed, non-negative cost associated with it, corresponding to the cost of the action.

Each node has a set of successor nodes. Corresponding to all operators (actions) that can apply at source node's state. Expanding a node is generating successor nodes and adding them (and associated arcs) to the state-space graph. One or more nodes may be designated as start nodes.

A **goal** test predicate is applied to a node to determine if its associated state is a goal state. A **solution** is a sequence of operations that is associated with a path in a state space from a start node to a goal node. The cost of a **solution** is the sum of the arc costs on the solution path.

State-space search is the process of searching through a state space for a solution by making explicit a sufficient portion of an implicit state-space graph to include a goal node.

Hence, initially $V = \{S\}$, where S is the start node; when S is expanded, its successors are generated, and those nodes are added to V , and the associated arcs are added to E . This process continues until a goal node is generated (included in V) and identified (by goal test).

To implement any **Uninformed search** algorithm, we always initialize and maintain a list called **OPEN** and put the start node of G in **OPEN**. If, after some time, we find **OPEN** is empty and we are not getting "goal node", then terminate with failure. We select a node n from **OPEN** and if $n \in \text{Goal node}$, then terminate with success, **else** we generate the successor of n (using operator O) and insert them in **OPEN**. In this way, we repeat the process till search is successful or unsuccessful.

Search strategies differ mainly in how to select an OPEN node for expansion at each step of the search.

A general search algorithm

1. **Initialize:** Set = {s}, where s is a start state.
2. **Fail:** If $OPEN = \{\}$, terminate with failure.
3. **Select:** Select a state, n, from OPEN
4. **Terminate:** If $n \in \text{Goal node}$, terminate with success
5. **Expand:** Generate the successor of n using operator O and insert them in OPEN.
6. **LOOP:** Goto Step 2

However, the problem with the above search algorithm is that it does not specify that a node should not be revisited once it has already been visited. That is “**how we can maintain a part of the state space that is already visited**”. So, we have an extension of the same algorithm, where we can save the explicit state space. To save the explicit space, we maintain another list called **CLOSED**.

Thus, to implement any Uninformed search algorithm efficiently, two lists, **OPEN** and **CLOSED**, are used.

Now we can select a node from OPEN and save it in CLOSED. The CLOSED list keeps a record of nodes that are Opened. The major difference between this algorithm and the previous one is that when we generate a successor node from CLOSED, we check whether it is already in $(OPEN \cup CLOSED)$. If it is already in $(OPEN \cup CLOSED)$, it is not inserted into OPEN again; otherwise, it is added to OPEN. The following modified algorithm is used to save the explicit space using the list CLOSED.

Modified search algorithm to save the explicit space

1. **Initialize:** Set $OPEN = \{s\}, CLOSED = \{\}$.
2. **Fail:** If $OPEN = \{\}$, terminate with failure.
3. **Select:** Select a state, n, from OPEN and save n in CLOSED
4. **Terminate:** If $n \in \text{Goal node}$, terminate with success
5. **Expand:** Generate the successor of n using operator O
For each successor, m, insert m in OPEN, only if $m \notin (OPEN \cup CLOSED)$
6. **LOOP:** Goto Step 2.

Here, the OPEN and CLOSED lists are used as follows:

OPEN: Nodes are yet to be visited.

CLOSED: Keeps track of all the nodes visited already

Note that initially OPEN list initializes with start state of G (e.g., **OPEN** = {*s*}) and CLOSED list as empty (e.g., **CLOSED** = {}).

Uninformed & Informed
Search

Insertion or removal of any node in OPEN depends on the specific search strategy.

3.2.1 Evaluation of Search Algorithms:

In any search algorithm, we select a node and generate its successor. Search strategies differ mainly in how an OPEN node is selected for expansion at each step of the search. Also, the insertion and deletion of nodes from the OPEN list depend on the specific search strategy. Any search algorithm is commonly evaluated according to the following four criteria, which are used to measure its performance:

- **Completeness:** Guarantees finding a solution whenever one exists.
- **Time Complexity:** How long (worst or average case) does it take to find a solution? Usually measured in terms of the number of nodes expanded.
- **Space Complexity:** How much space is used by the algorithm? Usually measured in terms of the maximum size that the "OPEN" list becomes during the search. Time and Space complexity are measured in terms of *b*: the branching factor or maximum number of successors of any node, *d*: the depth of the shallowest goal node (depth of the least cost solution), and *m*: the maximum depth (length) of any path in the state space (may be infinite).
- **Optimality/Admissibility:** If a solution is found, is it guaranteed to be an optimal one? For example, is it the one with the minimum cost?

The search process constructs a search tree, where **the root** is the initial state *S*, and the **leaf nodes** are nodes that have not yet been expanded (i.e., they are in OPEN List) or having no successors (i.e., they're dead ends). The search tree may be infinite because of loops, even if the state space is small. Search strategies mainly differ in how they select nodes from OPEN. Each node represents a partial solution path and its associated cost from the start node to the current node. In general, there are many possible paths from a given node and, therefore, many possible solutions that share this partial path as a prefix.

All search algorithms are distinguished by the order in which nodes are expanded. There are two broad classes of search methods: Uninformed search and Heuristic Search. Let us first discuss the Uninformed search.

3.3 UNINFORMED SEARCH

The uninformed search does not contain any domain knowledge, such as closeness or the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf

and goal nodes. Uninformed search explores the search tree without using any additional information about the search space, such as heuristic knowledge. It relies only on the initial state, the available operators, and the goal test; therefore, it is also called blind search. It examines each node of the tree until it achieves the goal node. Sometimes we may not get much relevant information to solve a problem.

For example, suppose we lost our car key, and we are not able to recall where we left it, we have to search for the key with some information, such as in which places we used to place it. It may be in our pants pocket or maybe in a table drawer. If it is not there, then we must search the whole house to get it. A reasonable approach would be to search the most likely locations first, such as the table drawer and then the wardrobe. In this situation, we must search with very few clues, making it an example of uninformed (blind) search.

Based on the order in which nodes are expanded, we have the following types of uninformed search algorithms:

- Breadth-first search
- Depth-first search
- Uniform cost search
- Iterative deepening depth-first search
- Bidirectional Search

3.3.1 Breadth-first search (BFS):

It is the simplest form of blind search. In this technique, the root node is expanded first, then all its successors are expanded, and then their successors, and so on. **In general, in BFS, all nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded.** It means that all immediate children of nodes are explored before any of the children's children are considered. The search tree generated by BFS is shown below in Fig 2.

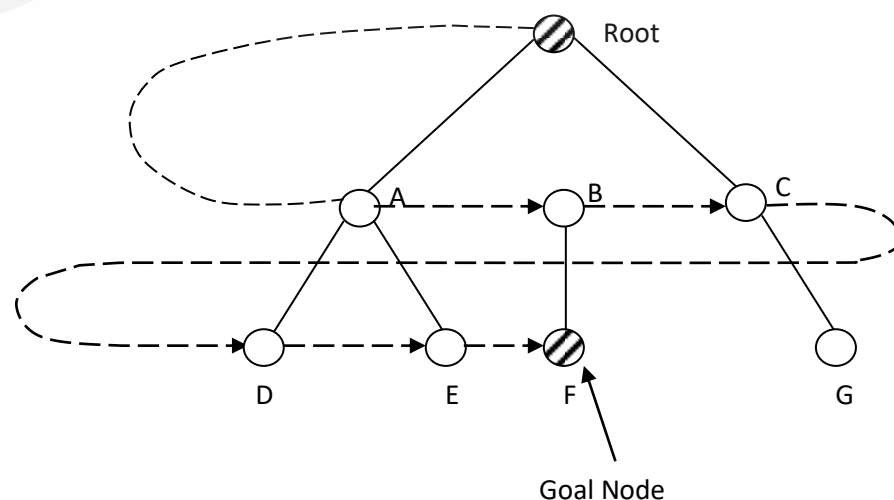


Fig 2 Search tree for BFS

Note that BFS is a brute-force search, so it generates all the nodes for identifying the goal, and note that we are using the convention that the alternatives are tried in the left-to-right order.

A BFS algorithm uses a data structure, a queue, that works on the FIFO principle. This queue will hold all generated but still unexplored nodes. **Please remember that the order in which nodes are placed on the queue, or removal and exploration, determines the type of search.**

We can implement it by using two lists called OPEN and CLOSED. The OPEN list contains those states that are to be expanded, and the CLOSED list keeps track of states already expanded. Here, the OPEN list is used as a **queue**.

BFS is effective when the search tree has a low branching factor.

Breath-First Search (BFS) Algorithm

- 1. Initialize:** Set = {s} , where s is a start state.
- 2. Fail:** If *OPEN* = { }, terminate with failure.
- 3. Select:** Remove the leftmost state (say *a*) from OPEN.
- 4. Terminate:** If *a* ∈ *Goal node*, terminate with success; else
- 5. Expand:** Generate the successor of node *a*, discard the successors of *a* if it's already in OPEN, insert only remaining successors on the right end of OPEN [i.e., QUEUE]
- 6. LOOP:** Goto Step 2

Let us take an example to see how this algorithm works.

Example 1: Consider the following graph in Figure 1 and its corresponding state space tree representation in Figure 2. Note that A is a start state and G is a goal state.

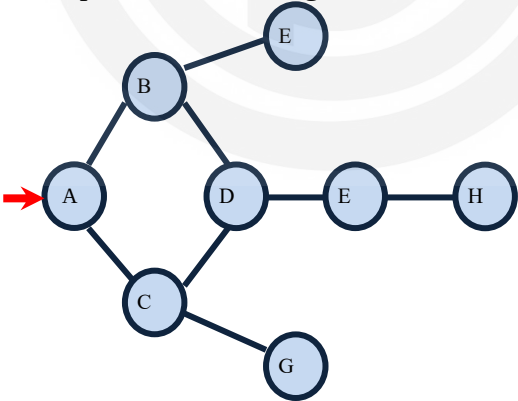


Fig-1 State space graph

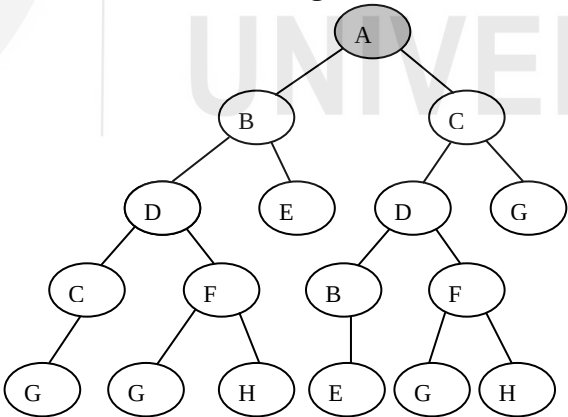
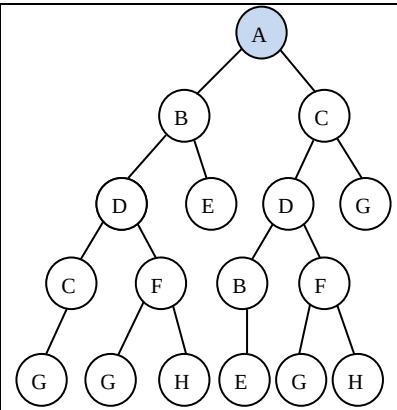
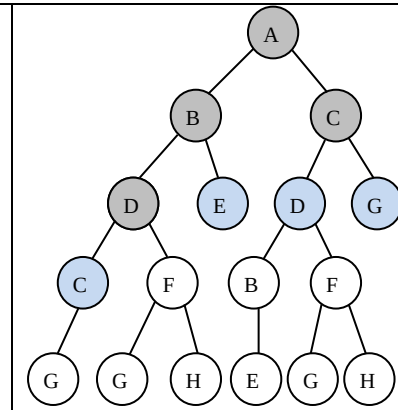
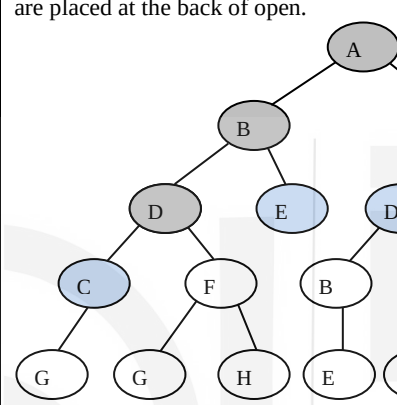
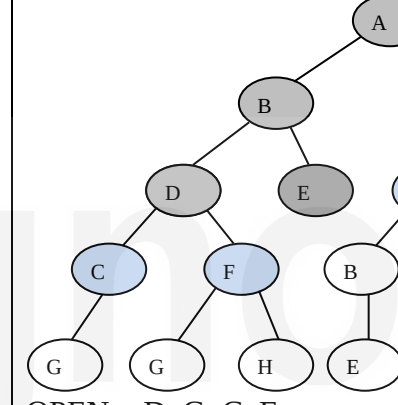
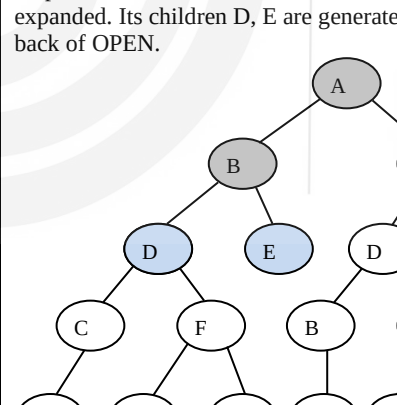
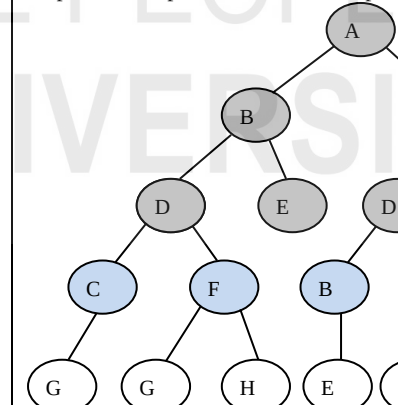
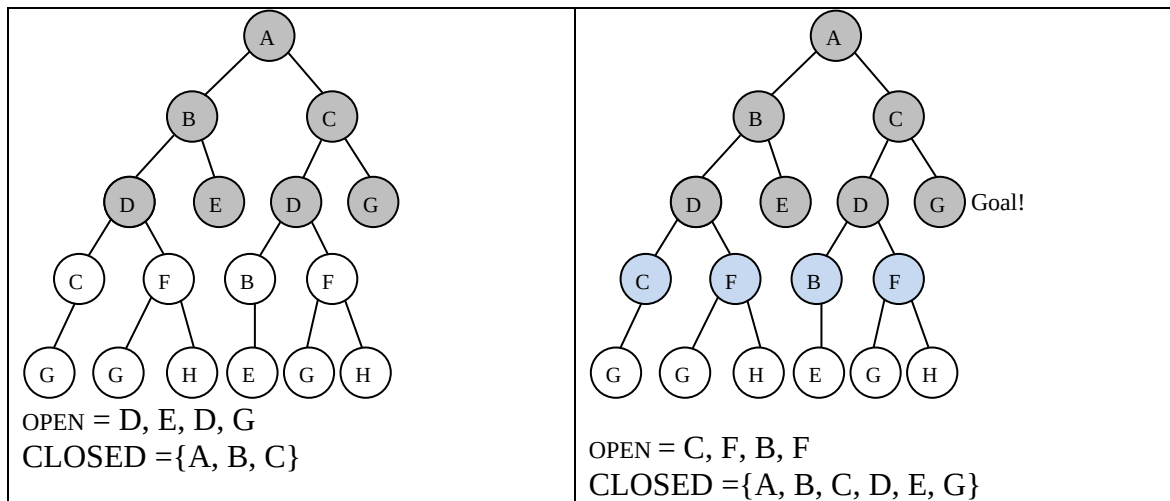


Fig-2 State space tree

Step1: Initially OPEN contains only one node corresponding to the source state A.	Step 5: Node D is removed from open. Its children C and F are generated and added to the back of OPEN.
--	---

 OPEN= A; CLOSED = { }	 OPEN = E, D, G, C, F CLOSED = A, B, C, D
Step2: A is removed from OPEN. The node is expanded, and its children B and C are generated. They are placed at the back of open.  OPEN = B, C CLOSED = {A }	Step 6: Node E is removed from OPEN. It has no children  OPEN = D, G, C, F CLOSED = A, B, C, D, E
Step: 3: Node B is removed from OPEN and is expanded. Its children D, E are generated and put at the back of OPEN.  OPEN = C, D, E ; CLOSED = {A, B}	Step 7: D is expanded, B and F are put in OPEN.  OPEN = G, C, F, B, F CLOSED = A, B, C, D, E, D
Step 4: Node C is removed from OPEN and is expanded. Its children D and G are added to the back of OPEN.	Step 8: G is selected for expansion. It is found to be a goal node. So, the algorithm returns the path ACG by following the parent pointers of the node corresponding to G. The algorithm terminates.



3.3.2 Time and Space complexity of BFS:

Consider a complete search tree of depth d , where each non-leaf node has b children (i.e., branching factor), and has a total of

$$1 + b + b^2 + b^3 + \dots + b^d = \frac{1(b^{d+1}-1)}{b-1} \text{ nodes.}$$

Time complexity is the number of nodes generated, so the time complexity of the BFS algorithm is $O(b^d)$

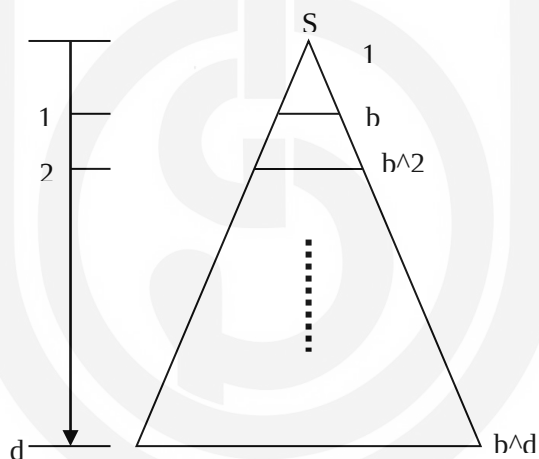


Fig 3: Search tree with b branching factor

For example, consider a complete search tree of depth 12, where every node at depths $0, 1, \dots, 11$ has 10 children (branching factor $b=10$) and every node at depth 12 has 0 children, then there are

$$1 + 10 + 10^2 + 10^3 + \dots + 10^{12} = \frac{1 \cdot (10^{13} - 1)}{10 - 1}$$

$$= \frac{10^{13} - 1}{9} = O(10^{12}) \text{ nodes in the complete search tree.}$$

- BFS is suitable for problems with shallow solutions

Space complexity:

BFS has to remember each and every node it has generated. Space complexity (maximum length of OPEN list):

So, space complexity is given by:

$$1 + b + b^2 + b^3 + \dots + b^d = O(b^d).$$

Performance of BFS:

- **Time** required for BFS for a tree of b branching factor and d depth is $O(b^d)$.
- **Space** (memory) requirement for a tree with b branching factor and d depth is also $O(b^d)$
- BFS algorithm is **Complete** (if b is finite).
- BFS algorithm is **Optimal** (if cost = 1 per step)

Space is the bigger problem in BFS as compared to DFS.

3.3.3 Advantages and disadvantages of BFS:

BFS has some advantages, which are given below

1. BFS will never get trapped exploring bund alley.
2. It is guaranteed to find a solution if one exists.

BFS has certain disadvantages, too. They are given below-

1. Time complexity and Space complexity are both $O(b^d)$, i.e., exponential type. This is a very big hurdle.
2. All nodes are to be generated in BFS. So, even unwanted nodes are to be remembered (stored in the queue), which is of no practical use in the search.

3.3.4 Depth First Search

A Depth-First Search (DFS) explores a path all the way to a leaf before backtracking and exploring another path. That is, expand the deepest unexpanded node (expand the most recently generated deepest node first).

The search tree generated by the DFS is shown in the figure below:

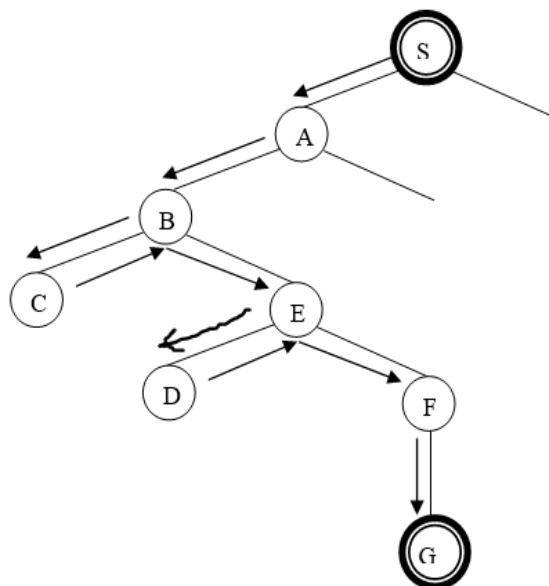


Fig. 4: Depth first search (DFS) tree

In depth-first search, we go as far down as possible into the search tree/graph before backing up and trying alternatives. It works by always generating a descendant of the most recently expanded node until some depth cut-off is reached, and then backtracks to the next most recently expanded node and generates one of its descendants. So, only the path of nodes from the initial node to the current node is stored, in order to execute the algorithm. For example, consider the following tree and see how the nodes are expanded using the DFS algorithm.

Example 1:

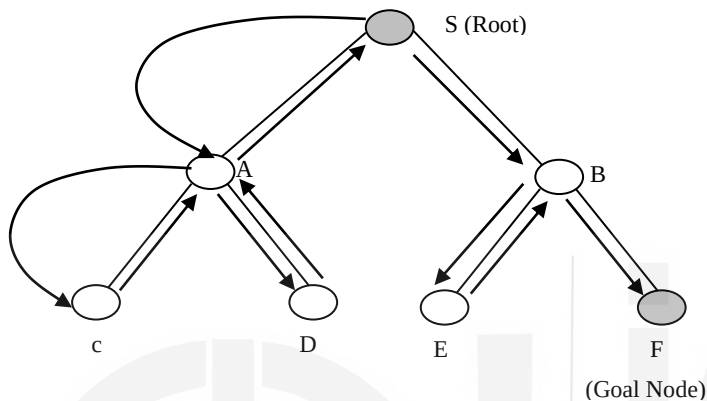


Fig. 5: Search tree for DFS

After searching root node S, then A and C, the search backtracks and tries another path from A. Nodes are explored in the order S, A, C, D, B, E, F .

Here again, we use the list OPEN as a STACK to implement DFS. If we found that the first element of OPEN is the goal state, then the search terminates successfully.

Depth-First Search (DFS) Algorithm

1. **Initialize:** Set = $\{s\}$, where s is a start state.
2. **Fail:** If $OPEN = \{\}$, terminate with failure.
3. **Select:** Remove the leftmost state (say a) from OPEN.
4. **Terminate:** If $a \in \text{Goal node}$, terminate with success, else
5. **Expand:** Generate the successor of the node a , discard the successors of a if it's already in OPEN, insert remaining successors on the **left** end of OPEN [i.e., STACK]
6. **LOOP:** Goto Step 2

Note: The only difference between BFS and DFS is in **Expand** (step 5). In BFS, we always insert the generated successors at the right end of the OPEN list, whereas in DFS at the left end of the OPEN list.

Properties of DFS Algorithm:

Suppose b (branching factor) is the maximum number of successors of any node, and m is the maximum depth of a leaf node, then

Number of nodes generated (in worst case): $1 + b + b^2 + \dots + b^m = O(b^m)$

3.3.5 Performance of DFS:

- **Time** required for DFS for a tree of b branching factor and m depth (of shallowest goal node) is $O(b^m)$.
- **Space** (memory) requirement for a tree with b branching factor and m depth (of shallowest goal node) is also $O(bm)$
- BFS algorithm is **Complete** (if b is finite).
- The BFS algorithm is not optimal.

Space is the advantage of DFS as compared to BFS.

3.3.6 Advantages and disadvantages of DFS:

Advantages:

- If depth-first search finds a solution without exploring much in a path, then the time and space it takes will be very less.
- The advantage of depth-first Search is that the memory requirement is only linear with respect to the search graph. This is in contrast with breadth-first search, which requires more space.

Example 1: Consider the following graph in Figure 1 and its corresponding state space tree representation in Figure 2. Note that A is a start state and G is a Goal state.

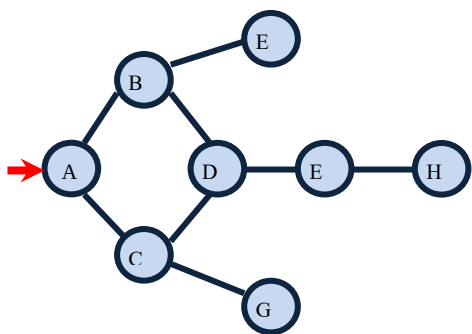


Fig. 1 State space graph

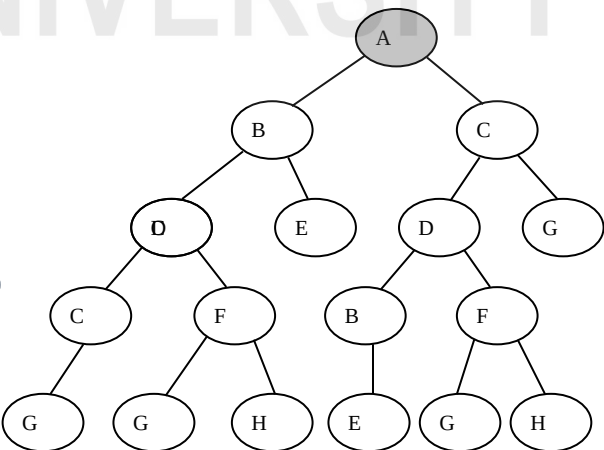
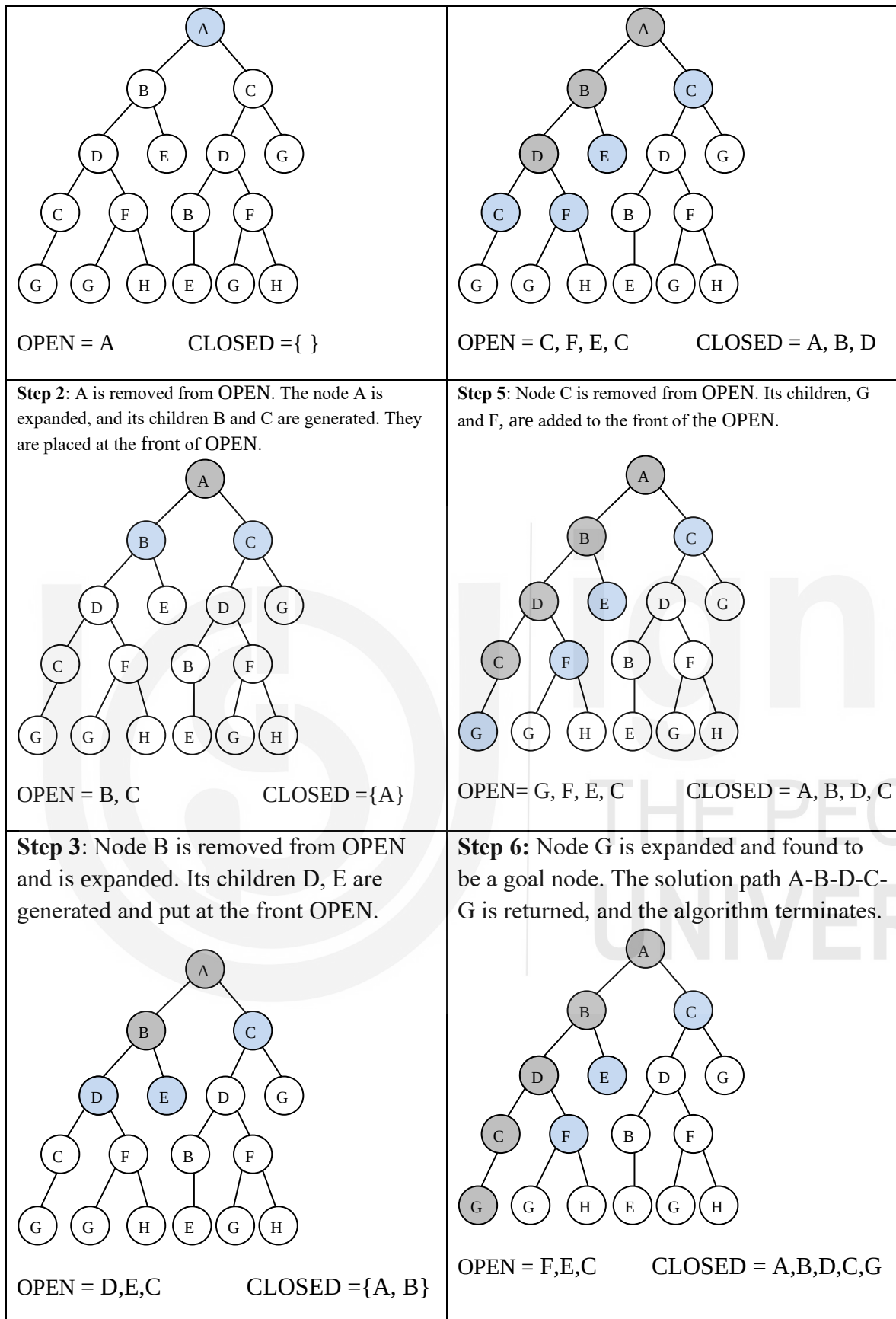


Fig. 2 State space tree

Step 1: Initially, the OPEN set contains only one node corresponding to the source state A.

Step 4: Node D is removed from OPEN and is expanded. Its children C and F are added to the front of the OPEN.



3.3.7 Comparison of BFS and DFS

BFS goes level wise, but requires more space as compared to DFS. The space required by DFS is $O(d)$, where d is the depth of the tree, but the space required by BFS is $O(b^d)$.

DFS: The problem with this approach is that if there is a node close to the root, but not in the first few subtrees explored by DFS, then DFS reaches that node very late. Also, DFS may not find the shortest path to a node (in terms of number of edges)

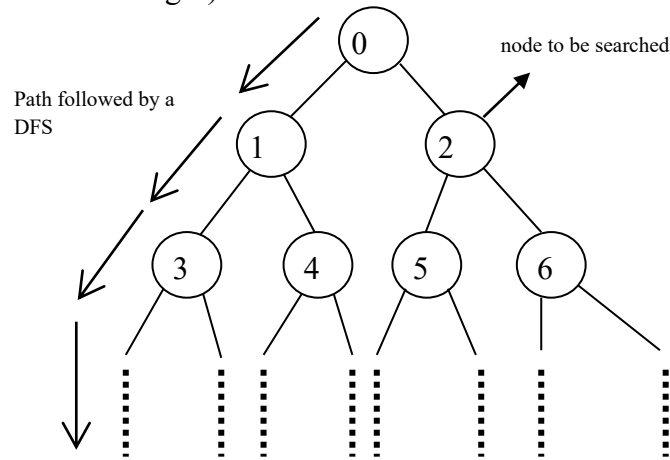


Fig 6: Path sequence in DFS

Suppose, we want to find node- '2' of the given infinite undirected graph/tree (Figure 6). A DFS starting from node-0 will dive left, towards node 1 and so on.

Whereas, node 2 is just adjacent to node 1.

Hence, a DFS wastes a lot of time coming back to node 2.

An **iterative deepening depth-first search** overcomes this and quickly finds the required node.

3.4 ITERATIVE DEEPENING DEPTH FIRST SEARCH (IDDFS)

Iterative Deepening Depth First Search (IDDFS) neither suffers the drawbacks of BFS nor DFS on trees. It takes advantage of both strategies.

It begins by performing DFS to a depth of zero, then to a depth of one, a depth of two, and so on until a solution is found or some maximum depth is reached.

It is like BFS in that it explores a complete layer of new nodes at each iteration before going to the next layer. It is like DFS for a single iteration.

It is preferred when there is a large search space, and the depth of a solution is not known. But it performs the wasted computation before reaching the goal depth. Since IDDFS expands all nodes at a given depth before expanding any nodes at greater depth, it is guaranteed to find a shortest-length (path) solution from the initial state to the goal state.

At any given time, it is performing a DFS and never searches deeper than depth 'd'. Hence, it uses the same space as DFS.

The disadvantage of IDDFS is that it performs wasted computation prior to reaching the goal depth.

Algorithm (IDDFS)

Initialized $d = 1$ /* depth of search tree */ , found = false

While (Found = False)

DO{

 perform a depth first search from start to depth d .

 if goal state is obtained

 then Found = true

else

discard the nodes generated in the search after depth d
(i.e. $d + 1$ onwards till last of tree)

}/* end while */

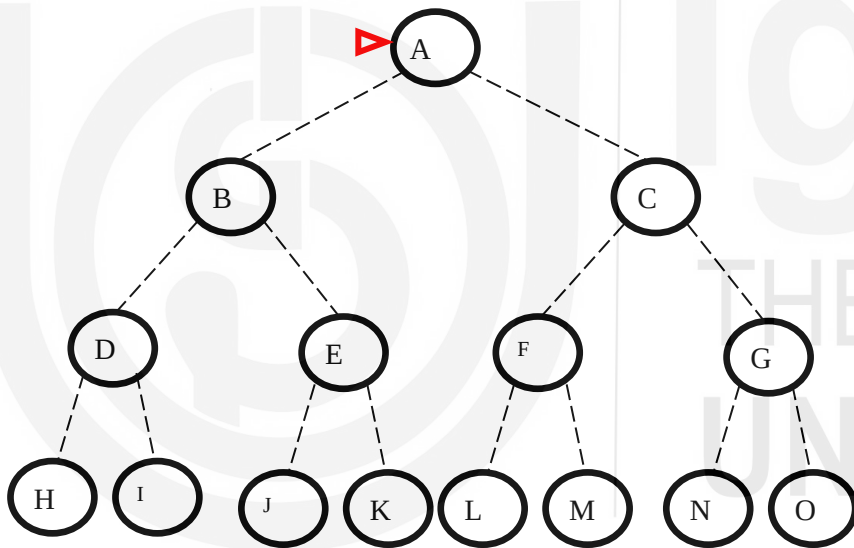
Report the solution, if obtained

Stop

Uninformed & Informed
Search

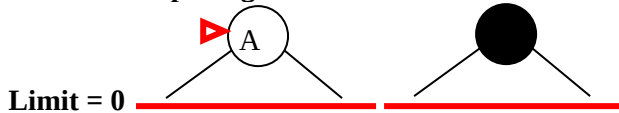
Let us consider the following example to understand the IDDFS:

- Here, the initial state is A and the goal state is M.

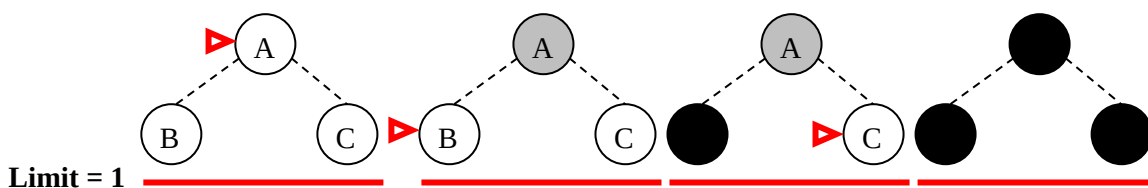


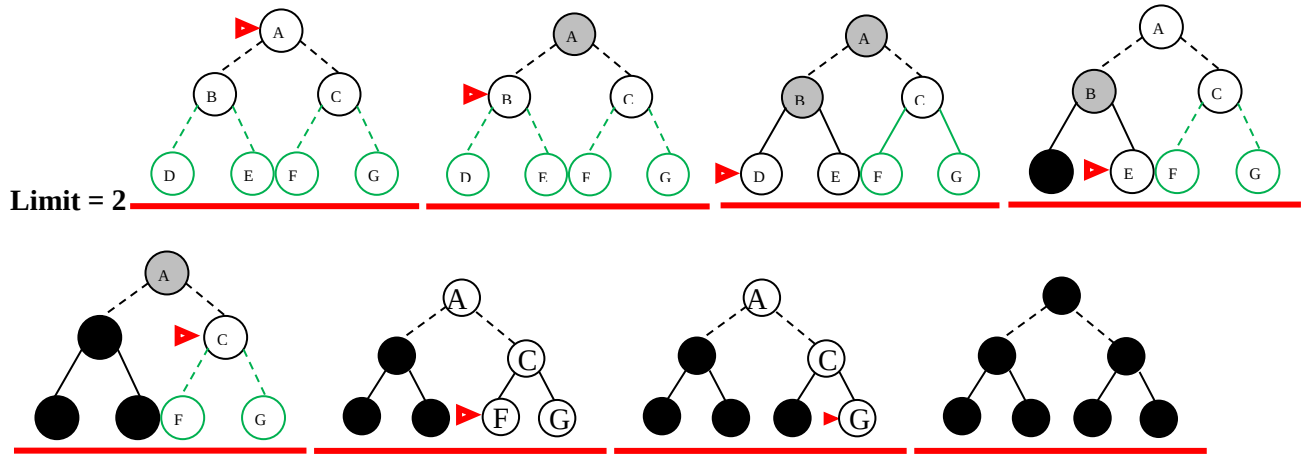
The Iterative deepening search proceeds as follows:

Iterative Deepening search $L = 0$

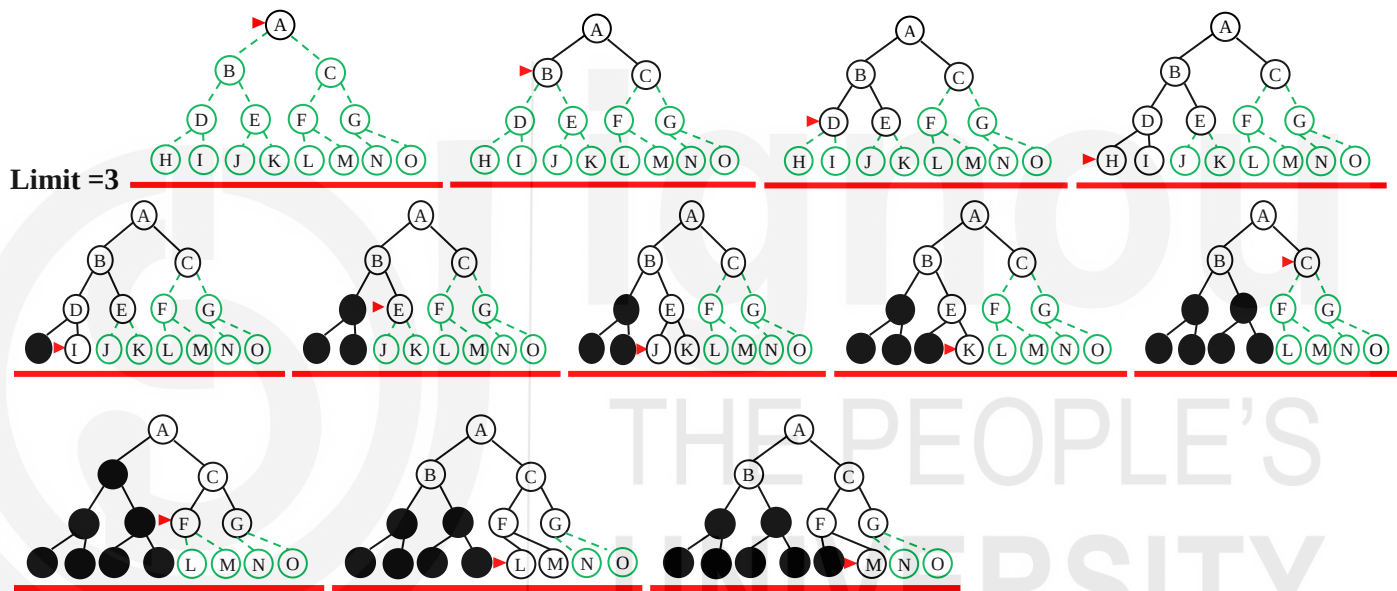


Iterative Deepening search $L = 1$





Iterative Deepening search $L = 3$



3.4.1 Time and space complexities of IDDFS

The time and space complexities of the IDDFS algorithm are $O(b^d)$ and $O(d)$, respectively.

It can be shown that depth-first iterative deepening is **asymptotically optimal**, among brute-force tree searches, in terms of time, space, and length of the solution. In fact, it is linear in its space complexity like DFS, and is asymptotically optimal to BFS in terms of the number of nodes expanded.

Please note that in general, iterative deepening is preferred over an uninformed search method when there is a large search space, and the depth of the solution is unknown. Also note that iterative deepening search is analogous to BFS in that it explores a complete layer, going to the next layer.

3.4.2 Advantages and Disadvantages of IDDFS:

Advantages:

1. It combines the benefits of BFS and DFS search algorithms in terms of fast search and memory efficiency.
2. It is guaranteed to find the shortest path solution.
3. It is a preferred uniformed search method when the search space is large and the depth of the solution is not known.

Disadvantages

1. The main drawback of IDDFS is that it repeats all the work from the previous phase, that is, it performs wasted computations before reaching the goal depth.
2. The time complexity is $O(b^d)$ i.e., exponential type only.

The following summarizes when to use which algorithm:

DFS	BFS	IDDFS
Many solutions exist. Know (or have a good estimate of) the depth of solution.	Some solutions are known to be shallow	Space is limited and the shortest solution path is required

3.5 BIDIRECTIONAL SEARCH

This search technique expands nodes from the start and goal state simultaneously. Check at each stage if the nodes of one have been generated by the other. If so, the path concatenation is the solution.

- This search is used when a problem has a single goal state that is given explicitly, and all the node generation operators have inverses.
- So, it is used to find the shortest path from an initial node to a goal node, instead of finding the goal itself along with the path.
- It works by searching forward from the initial node and backward from the goal node simultaneously, by hoping that the two searches meet in the middle.
- Check at each stage if the nodes of one have been generated by the other, i.e., they meet in the middle.
- If so, the path concatenation is the solution.

Thus, the BS Algorithm is applicable when generating predecessors is easy in both forward and backward directions, and there exist only one or fewer goal states. The following figure illustrates how the Bidirectional search is executed.

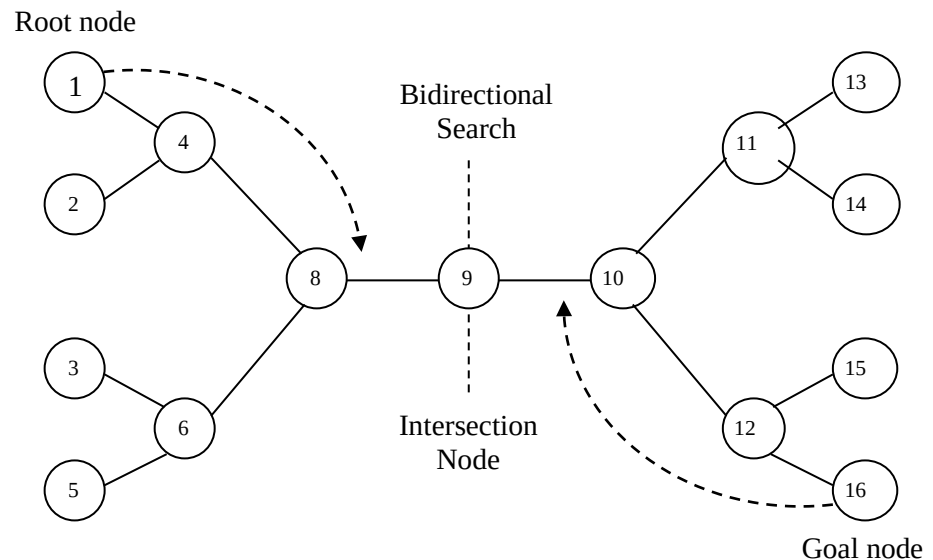


Fig. 7: Bidirectional search

We have node 1 as the start/root node and node 16 as the goal node. The algorithm divides the search tree into two sub-trees. So, from start node 1, we do a forward search, and at the same time, we do a backward search from goal node 16. The forward search traverses nodes 1, 4, 8, and 9, whereas the backward search traverses through nodes 16, 12, 10, and 9. We see that both the forward and backward searches meet at node 9, called the intersection node. So, the total path traced by the forward search and the path traced by the backward search is the optimal solution. This is how the BS Algorithm is implemented.

Advantages:

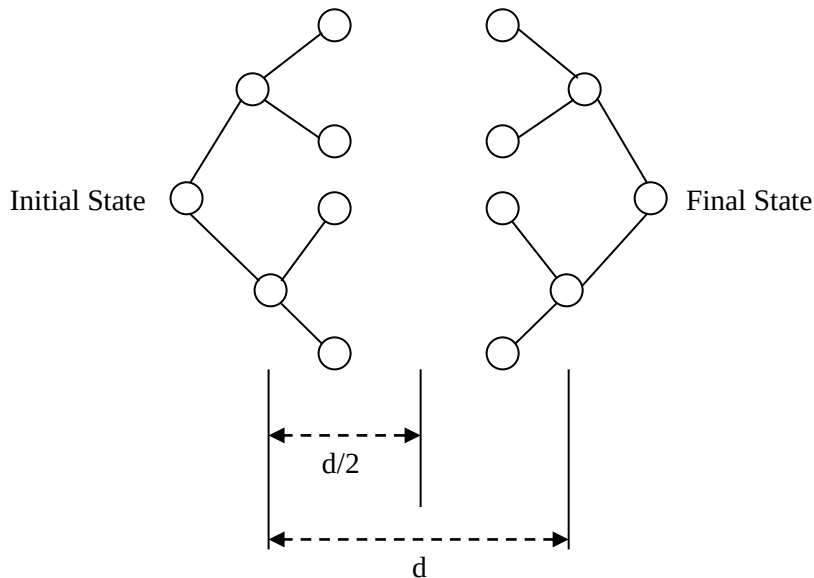
- Since BS uses various techniques like DFS, BFS, Depth-limited search (DLS), etc., it is efficient and requires less memory.

Disadvantages:

- The implementation of the bidirectional search tree is difficult.
- In bidirectional search, one should know the goal state in advance.
- It is practically inefficient due to the additional overhead of performing the insertion operation at each point of the search.

Time complexity:

The total number of nodes expanded in bidirectional search is $= 2b^{d/2} = O(b^{d/2})$, where b is a branching factor, and d is the depth of the shallowest goal node.



Finally, the Bidirectional search is:

- Complete? Yes
- Time Complexity: $O(b^{d/2})$
- Space complexity: $O(b^{d/2})$
- Optimal: Yes (if step cost is uniform in both forward and backward directions)

3.6 COMPARISON OF UNINFORMED SEARCH STRATEGIES

The following Table 1 compares the efficiency of uninformed search algorithms. These are the measures to evaluate the performance of the search algorithms:

Table -1 Performance of uninformed search algorithm

	BFS	DFS	IDDFS	Bidirectional Search (if applicable)
Time	b^d	b^d	b^d	$b^{d/2}$
Space	b^d	bm	bd	$b^{d/2}$
Optimum?	Yes	No	Yes	Yes
Complete?	Yes	No	Yes	Yes

Where

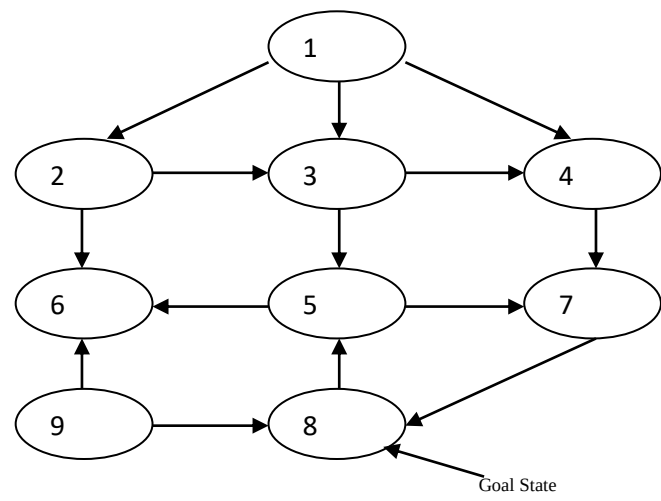
b =branching factor

d =depth of shallowest goal state

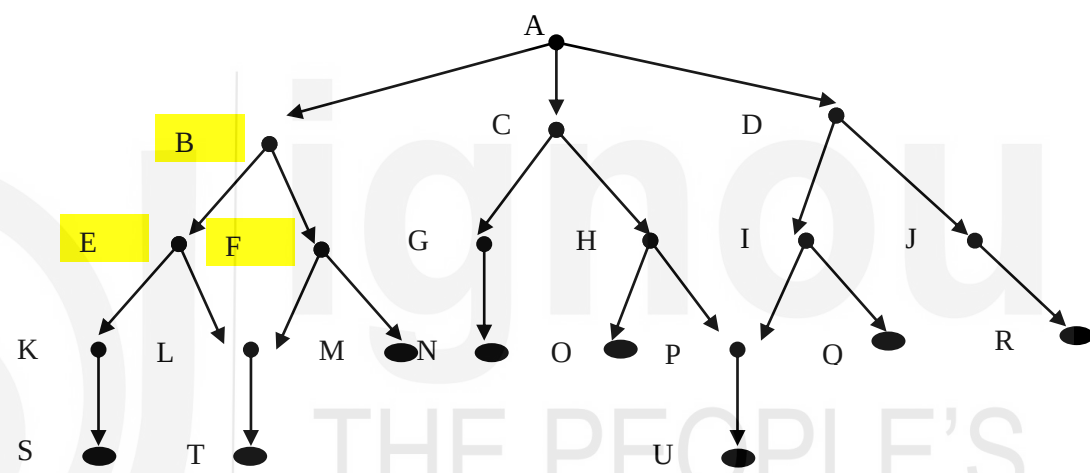
m =Maximum depth of the search space

☞ Check Your Progress 1

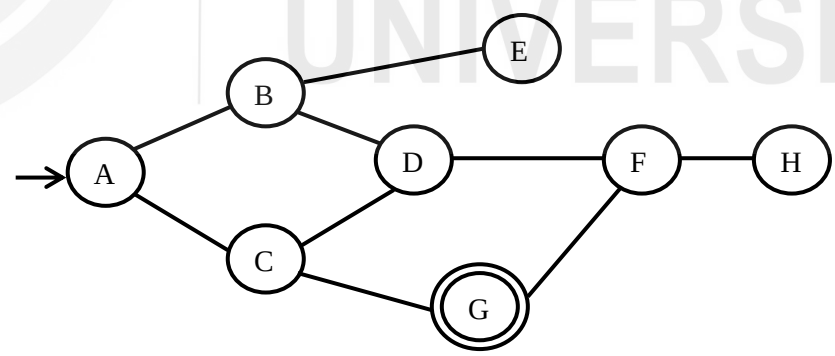
Q.1 Apply BFS and DFS algorithms on the following graph, clearly showing the contents of OPEN and CLOSE lists.



Q.2 Apply BFS algorithm on the following tree (M is the goal node)



Q.3 Apply BFS algorithm on the following graph.



Let A be the state, and G be the final or goal state to be searched.

Q.4 Compare the Uninformed search algorithm with respect to Time, Space, Optimal, and Complete.

3.7 INFORMED (HEURISTIC) SEARCH

Uninformed (blind) search is inefficient in most cases because they do not have any domain-specific knowledge about the goal state. Heuristic Search

uses domain-dependent (heuristic) information beyond the definition of the problem itself in order to search the space more efficiently.

Uninformed & Informed
Search

The following are some ways of using heuristic information:

- Deciding which node to expand next, instead of doing the expansion in a strictly breadth-first or depth-first order;
- In the course of expanding a node, deciding which successor or successors to generate, instead of blindly generating all possible successors at one time;
- Deciding that certain nodes should be discarded, or pruned, from the search space.

Informed search algorithms use domain knowledge. In an informed search, problem information is available, which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a **Heuristic search**.

Heuristics are a guesswork, or additional information about the problem. It may miss the solution if wrong heuristics are supplied. However, in almost all problems with correct heuristic information, it provides a good solution in a reasonable time

Informed search can solve many complex problems that could not be solved in any other way.

We have the following informed search algorithm:

1. Best-First Search
2. A* algorithm
3. Iterative Deepening A*

3.7.1 Strategies for providing heuristics information:

The informed search algorithm is more useful for a large search space. An informed search algorithm uses the idea of a heuristic, so it is also called a heuristic search.

“Heuristics are criteria, methods, or principles for deciding which among several alternative courses of action promises to be the most effective to achieve some goal.”

Heuristic Function:

Heuristic information is provided in the form of a function called **the heuristic function**.

- ❖ A heuristic is a function that is used in Informed Search, and it finds the most promising path.
- ❖ It takes the current state of the agent as its input and produces the estimation of how close the agent is to the goal.
- ❖ Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states.
- ❖ The heuristic method, however, might not always give the best solution,

but it is guaranteed to find a good solution in a reasonable time.

- ❖ This technique is always used to find a solution quickly.
- ❖ Informed Search defines a heuristic function $h(n)$ that estimates the "goodness" of a node n .
- ❖ The heuristic function is an estimate, based on domain-specific information that is computable from the current state description of how close we are to a goal.
- ❖ Specifically, $h(n)$ = estimated cost (or distance) of the minimal cost path from state 'n' to a goal state.

A heuristic function at a node n is an estimate of the optimum cost from the current node to a goal, denoted by $h(n)$

$h(n)$ = estimated cost of the cheapest path from node n to a goal node

For example, suppose you want to find the shortest path from Kolkata to Guwahati, then the heuristic for Guwahati may be the straight-line distance between Kolkata and Guwahati, that is:

$$h(\text{Kolkata}) = \text{euclideanDistance}(\text{Kolkata}, \text{Guwahati})$$

3.7.2 Formulation of informed (heuristic) search problem as State Space

Informed search problems can be represented as state space. The state space of a problem includes: an Initial state, one or more goal states, a set of state transition operators O (a set of rules), used to change the current state to another state, and a heuristic function h .

In general, a state space is represented by 5-tuples as follows: $S: [S, S_0, O, G, h]$
Where S : (Implicitly specified) Set of all possible states (possibly infinite).

S_0 : start state of the problem, $S_0 \in S$.

O : Set of state transition operators, each having the same cost. This is used to change the state from one state to another. It is the set of arcs (or links) between nodes

G : Set of Goal states, $G \subseteq S$.

$h()$: A heuristic function, estimating the distance to a goal node.

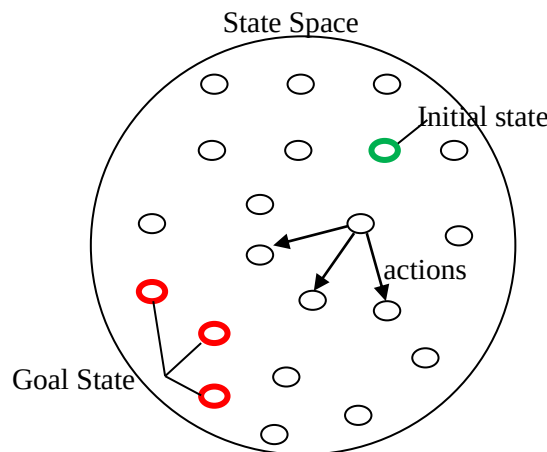


Fig. 8: State space with initial and goal node

We need to find a sequence of actions that transforms the agent from the initial state S_0 to Goal state G . State space is commonly defined as a directed graph or as a tree in which each node is a state, and each arc represents the application of an operator transforming a state to a successor state.

Thus, the problem is solved by using the rules (operators), in combination with an appropriate control strategy, to move through the problem space until a path from the **initial state to a goal state** is found. This process is known as **search**. A solution path is a path in the state space from S_0 (initial state) to G (Goal state).

We have already seen that an OPEN list is used to implement an uninformed (Blind) search (section 3.2). But the problem with using only one list OPEN is that it is not possible to keep track of the node that has already been visited. That is “**how we can maintain a part of the state space that is already visited**”. To save the explicit space, we maintain another list called **CLOSED**. Now we can select a node from OPEN and save it in **CLOSED**. Now, when we generate a successor node from CLOSED, we check whether it is already in $(OPEN \cup CLOSED)$. If it is already in $(OPEN \cup CLOSED)$, we will not insert it into OPEN again, otherwise insert it.

3.7.3 Best-First Search

Best first search uses an evaluation function $f(n)$ that gives an indication of which node to expand next for each node. Every node in a search space has an evaluation function (heuristic function) associated with it. A heuristic function value $h(n)$ on each node indicates how the node is from the goal node. Note that the evaluation function = heuristic cost function (in case of minimization problem) OR objective function (in case of maximization). The decision of which node to be expanded depends on the value of the evaluation function. Evaluation value = cost/distance of current node from goal node, and for the goal node evaluation function value=0

Based on the evaluation function, $f(n)$, Best-first search can be categorized into the following categories:

- 1) Greedy Best first search
- 2) A* search

The following 2 list (**OPEN** and **CLOSED**) are maintained to implement these two algorithms.

1. **OPEN** – all those nodes that have been generated & have had a heuristic function applied to them but have not yet been examined.
2. **CLOSED**- contains all nodes that have already been examined.

3.7.4 Greedy Best-First Search:

The greedy best-first search algorithm always selects the path that appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms. It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms. With the help of best-first

search, at each step, we can choose the most promising node. In the best first search algorithm, we expand the node that is closest to the goal node, and the closest cost is estimated by the heuristic function, i.e.

$$f(n) = g(n).$$

Where $h(n)$ = estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the **priority queue** (or to store the heuristic function value).

Best first search algorithm:

1. Initialize: Set $OPEN = \{s\}$, $CLOSED = \{ \}$;

$$f(s) = h(s)$$

2. Fail: If $OPEN = \{ \}$, terminate with failure.

3. Select: Select the minimum cost state n from $OPEN$, save n in $CLOSED$.

4. Terminate: If $n \in Goal\ node$, terminate with success and return $f(n)$, else

5. Expand: For each successor, m of n ,

If $m \notin [OPEN \cup CLOSED]$

Set $f(m) = h(m)$ and Insert m in $OPEN$

.....

6. LOOP: Goto Step 2

OR Pseudocode (for Best-First Search algorithm)

Step 1: Place the starting node into the $OPEN$ list.

Step 2: If the $OPEN$ list is empty, stop and return failure.

Step 3: Remove the node n from the $OPEN$ list that has the lowest value of $h(n)$, and place it in the $CLOSED$ list.

Step 4: Expand the node n , and generate the successors of node n .

Step 5: Check each successor of node n , and find whether any node is a goal node or not. If any successor node is a goal node, then return success and terminate the search; else, proceed to Step 6.

Step 6: For each successor node, the algorithm checks for the evaluation function $f(n)$, and then checks if the node has been in either $OPEN$ or the $CLOSED$ list. If the node has not been in either list, then add it to the $OPEN$ list.

Step 7: Return to Step 2.

Advantages of Best-First search:

- The best first search can switch between BFS and DFS, thus gaining the advantages of both algorithms.
- This algorithm is more efficient than the BFS and DFS algorithms.

Disadvantages of Best-First search:

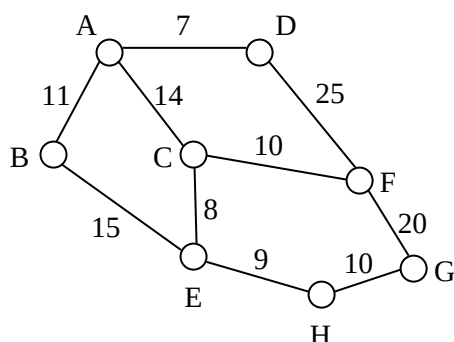
- Chances of getting stuck in a loop are higher.
- It can behave as an unguided depth-first search in the worst-case scenario.

Consider the following example for a better understanding of the greedy Best-First search algorithm.

Example 1: Consider the following **example** (graph) with heuristic function value $h(n)$, which illustrates the greedy Best-first search. Note that in the following example, the heuristic function is defined as

Uninformed & Informed
Search

$$h_{SLD} = \text{straight line distance from } n \text{ to goal}$$



Let the heuristic function value $h(n)$ for each node n to the goal node G be defined as:

Straight line distance

$$A \rightarrow G = h(A) = 40$$

$$B \rightarrow G = h(B) = 32$$

$$C \rightarrow G = h(C) = 25$$

$$D \rightarrow G = h(D) = 35$$

$$E \rightarrow G = h(E) = 19$$

$$F \rightarrow G = h(F) = 17$$

$$H \rightarrow G = h(H) = 10$$

$$G \rightarrow G = h(G) = 0$$

$h(n)$ = straight line distance from node n to G

Note that $h(G)=0$

The nodes added/deleted from OPEN and CLOSED lists using Best-First Search algorithm are shown below.

OPEN	CLOSED
[A]	[]
[C, B, D]	[A]
B, D	A, C
F, E, B, D	A, C
G, E, B, D	A, C, F
E, B, D	A, C, F, G

Explanations are as follows:

Step 1: Initially, OPEN list starts with start state 'A', and CLOSED list is empty.

Step2: Children of A={C[25], B[32], D[35]}, so

OPEN={C[25], B[32], D[35]} therefore Best = C, so expand C node next.

Step3: Children of C={E[19], F[17]}, so

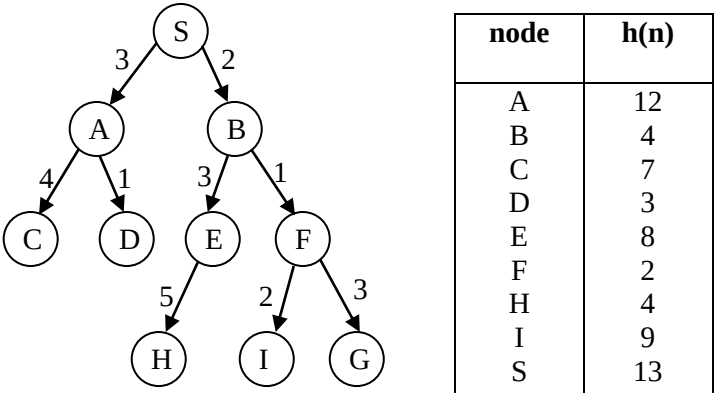
OPEN={F[17], E[19], B[32], D[35]} therefore Best = F, so expand node F next.

Step4: Children of F = {G[0]}, therefore OPEN= {G[0], E[19], B[32], D[35]}

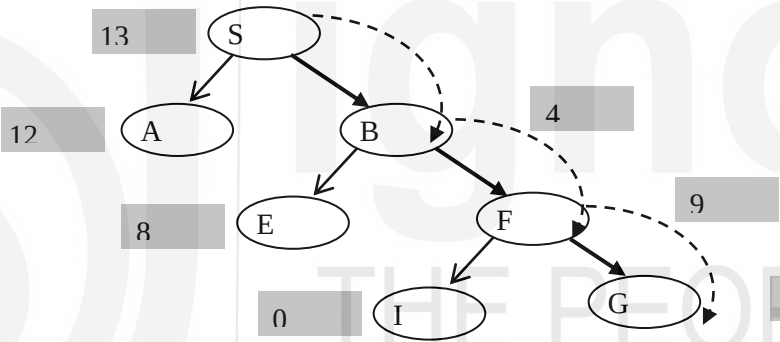
Best=G, this is a goal node so Stop.

Finally, we got the shortest path: $A \rightarrow C \rightarrow F \rightarrow G$, and the cost is 44.

Example 2: Consider the search problem below, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using the evaluation function $f(n)=h(n)$, which is given in the table below.



Here, we are using two lists, which are the **OPEN** and **CLOSED** Lists. The following are the iterations for traversing the above example.



Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration2: Open [E, F, A], Closed [S, B]

: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]

: Open [I, E, A], Closed [S, B, F, G]

Hence, the final solution path will be $S \rightarrow B \rightarrow F \rightarrow G$

Evaluation of Best-First Search algorithm:

Time Complexity:

The worst-case time complexity of Greedy best-first search is $O(b^m)$.

Space Complexity:

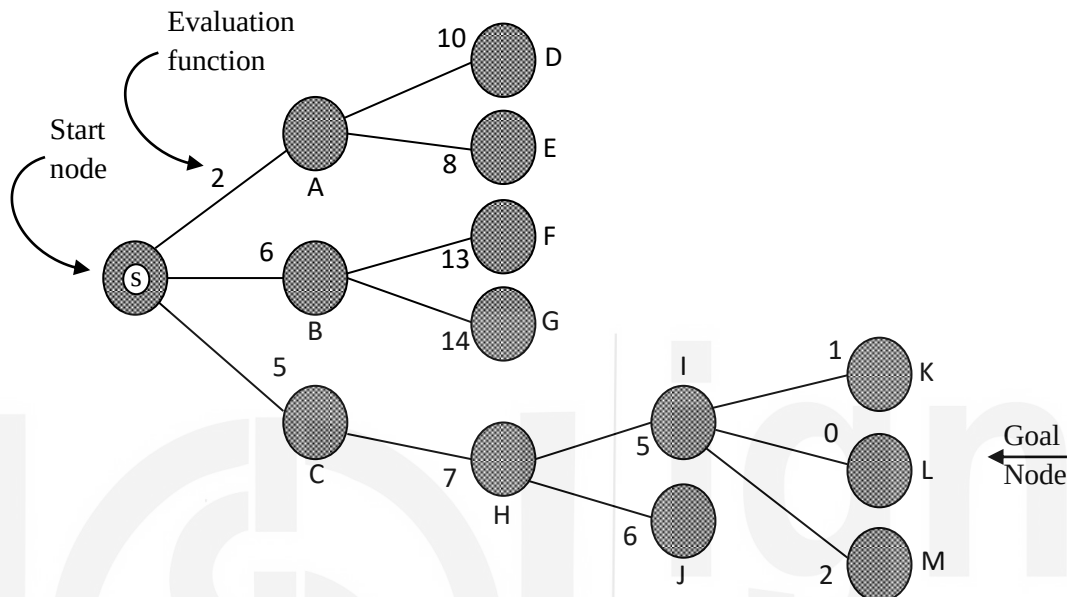
The worst-case space complexity of Greedy best-first search is $O(b^m)$. Where m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Uninformed & Informed Search

Optimal: Greedy best first search algorithm is not optimal.

Example 3: Apply the Greedy Best-First Search algorithm on the following graph (L is a goal node).



Working: We start with a start node, S. Now, S has three children, i.e., A, B, and C with their Heuristic function values 2, 6, and 5, respectively. These weights show approximately how far they are from the goal node. So, we write, children of S are: (A: 2), (B: 6), (C: 5)

Out of these, the node with the minimum value is (A: 2). So, we select A, and its children are explored (or generated).

Its children are (D: 10) and (E: 8)

The search process now has four nodes to search for, namely-

(B: 6), (C: 5), (D: 10), and (E: 8)

Out of these, node-C has the minimal value of 5. So, we select it and expand.

So, we get (H: 7) as its child. Now, the nodes to search are as follows-

(B:6), (D: 10), (E: 8), (H: 7), and so on.

Working of the algorithm can be represented in a tabular form as follows:

Step	Node being expanded	Children (on expansion)	Available nodes (to search)	Node Chosen
1.	S	(A:2), (B:6), (C:5)	(A:2), (B:6), (C:5)	(A:2)
2.	A	(D:10), (E:8)	(B:6), (C:5), (D:10), (E:8)	(C:5)
3.	C	(H:7)	(B:6), (D:10), (E:8), (H:7)	(B:6)
4.	B	(F:13), (G:14)	(D:10), (E:8), (H:7), (F:13), (G:14)	(H:7)
5.	H	(I:5), (J:6)	(D:10), (E:8), (F:13), (G:14), (I:5), (J:6)	(I:5)

6.	I	(K:1), (L:0), (M:2)	(D:10), (E:8), (H:7), (F:13), (G:14), (J:6), (K:1), (L:0), (M:2)	Goal node is found. So, search stops now
----	---	---------------------	---	--

3.8 A* ALGORITHM

A* search is the most commonly known form of best-first search. It uses a heuristic function $h(n)$ and the cost to reach node n from the start state $g(n)$. It has combined features of uniform cost search (UCS) and greedy best-first search, by which it solves the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands a smaller search tree and provides an optimal result faster.

In A* search algorithm, we use a search heuristic as well as the cost to reach the node. Hence, we can combine both costs as follows, and this sum is called the **fitness number (Evaluation Function)**.

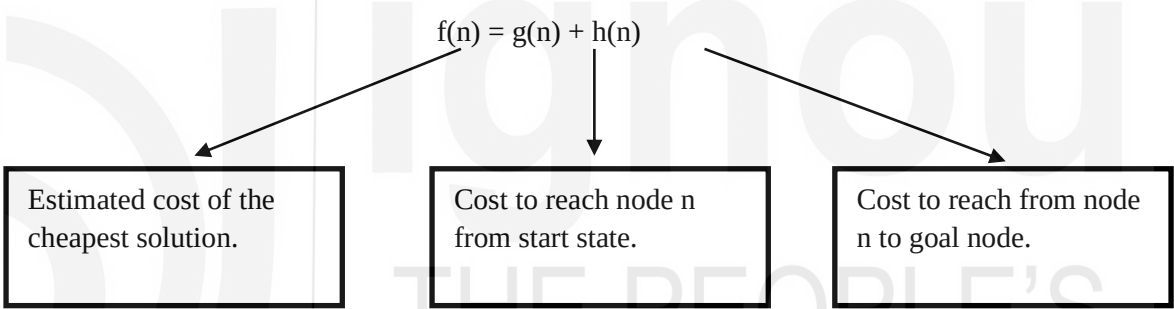


Fig 9: Evaluation function $f(n)$ in A* Algorithm

A* algorithm **evaluation function $f(n)$** is defined as **$f(n) = g(n) + h(n)$**
Where **$g(n)$** =sum of edge costs from the start state to n
And **$h(n)$** = An estimate of the lowest cost path from node $n \rightarrow$ goal node.

If $h(n)$ is **admissible**, then search will find an optimal solution. Admissible means underestimates the cost of any solution that can be reached from a node. In other words, a heuristic is called admissible if it always **underestimates**, that is, we always have $h(n) \leq h^*(n)$, where $h^*(n)$ denotes the minimum distance to a goal state from state n .

A* search begins at root node, and then the search continues by visiting the next node which has the least evaluation value $f(n)$.

It evaluates nodes by using the following evaluation function
 $f(n) = h(n) + g(n)$ = estimated cost of the cheapest solution through n .

Where,

$g(n)$: the actual shortest distance traveled from the initial node to the current node, which helps to avoid expanding paths that are already expansive
 $h(n)$: the estimated (or “heuristic”) distance from the current node to the goal, it estimates which node is closest to the goal node.
Nodes are visited in this manner until a goal is reached.

Suppose s is a start state, then the calculation of the evaluation function $f(n)$ for any node n is shown in the following figure 10.

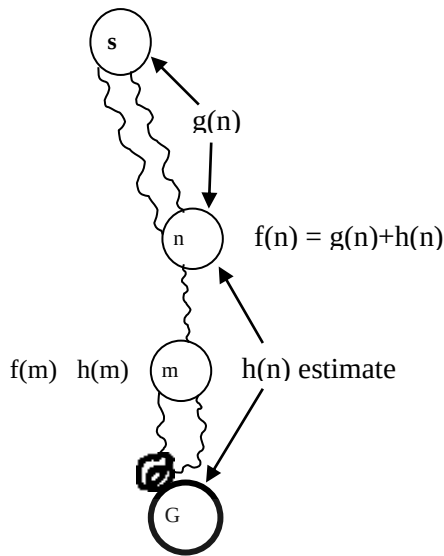


Fig 10 Calculation of evaluation function $f(n)$.

Algorithm A*

1. **Initialize:** Set $OPEN = (s)$, $CLOSED = \{\}$, $g(s) = 0$, $f(s) = h(s)$
2. **Fail:** If $OPEN = \{\}$, Terminate & fail
3. **Select :** Select the minimum cost state, n , from $OPEN$, Save n in $CLOSED$.
4. **Terminate:** If $n \in G$, terminate with success, and return $f(n)$
5. **Expand:** For each successor, m , of n
 - If $m \notin [OPEN \cup CLOSED]$
 - Set $g(m) = g(n) + C(n, m)$
 - Set $f(m) = g(m) + h(m)$
 - Insert m in $OPEN$
 - If $m \in [OPEN \cup CLOSED]$
 - Set $g(m) = \min \{ \min \{ g(m), g(n) + C(n, m) \} \}$
 - Set $f(m) = g(m) + h(m)$
 - If $f(m)$ has decreased and $m \in CLOSED$, move m to $OPEN$

In step 5, we generate a successor of n (say m), and for each successor m , if it does not belong to $OPEN$ or $CLOSED$, that is $m \notin [OPEN \cup CLOSED]$, then we insert it into $OPEN$ with the cost $g(n) + C(n, m)$, i.e., cost up to n and additional cost from $n \rightarrow m$.

If $m \in [OPEN \cup CLOSED]$ then we set $g(m)$ with the original cost and the new cost $[g(n) + C(n, m)]$. If we arrive at some state with another path that has less cost than the original one, then we replace the existing cost with this minimum cost.

If we find that $f(m)$ is decreased (if larger then ignore) and $m \in CLOSED$ then move m from $CLOSED$ to $OPEN$.

Note that the implementation of A* Algorithm involves maintaining two lists- **OPEN** and **CLOSED**. The list **OPEN** contains those nodes that have been evaluated by the heuristic function but have not expanded into successors yet and the list **CLOSED** contains those nodes that have already been visited. See the following steps for working of A* algorithm:

Step-1: Define a list OPEN. Initially, OPEN consists of a single node, the start node S.

Step-2: If the list is empty, return failure and exit.

Step-3: Remove node n with the smallest value of $f(n)$ from OPEN and move it to list CLOSED.

If node n is a goal state, return success and exit.

Step-4: Expand node n.

Step-5: If any successor to n is the goal node, return success and the solution by tracing the path from the goal node to S.

Otherwise, go to Step 6.

Step-6: For each successor node,

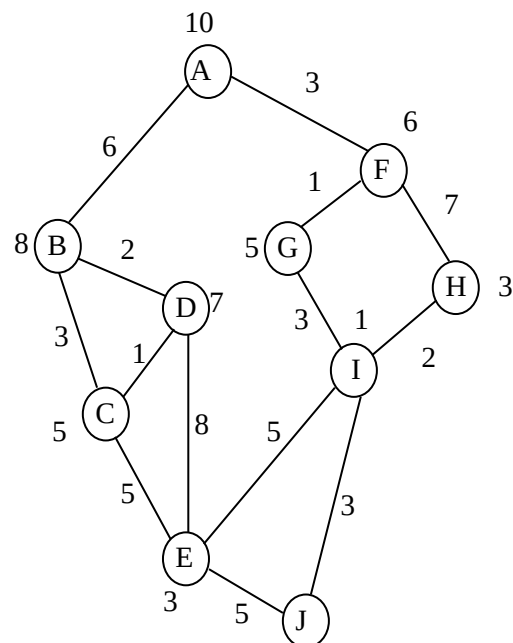
Apply the evaluation function f to the node.

If the node has not been in either list, add it to OPEN.

Step-7: Go back to Step-2.

3.8.1 Working of A* algorithm

Example 1: Let's us consider the following graph to understand the working of A* algorithm. The numbers written on edges represent the distance between the nodes. The numbers written on nodes represent the heuristic value. Find the most cost-effective path to reach from start state **A** to **final** state **J** using A* Algorithm.



Step-1:

We start with node A. Node B and Node F can be reached from node A. A* Algorithm calculates $f(B)$ and $f(F)$. Estimated Cost $f(n) = g(n) + h(n)$ for Node B and Node F is:

$$f(B) = 6+8=14$$

$$f(F) = 3+6=9$$

Since $f(F) < f(B)$, it decides to go to node F.

➔ Closed list (F)

Path- $A \rightarrow F$

Step-2: Node G and Node H can be reached from Node F.

A* Algorithm calculates $f(G)$ and $f(H)$.

$$f(G) = (3+1)=5=9$$

$$f(H) = (3+7) +5=13$$

Since $f(G) < f(H)$, it decides to go to node G.

➔ Closed list (G)

Path- $A \rightarrow F \rightarrow G$

Step-3: Node I can be reached from node G.

A* Algorithm calculates $f(I)$.

$$f(I) = (3+1+3)+1=8; \text{ It decides to go to node I.}$$

➔ Closed list (I).

Path- $A \rightarrow F \rightarrow G \rightarrow I$

Step-4: Node E, Node H, and Node J can be reached from Node I.

A* Algorithm calculates $f(E)$, $f(H)$, $f(J)$.

$$f(E) = (3+1+3+5) + 3 = 15$$

$$f(H) = (3+1+3+2) + 3 = 12$$

$$f(J) = (3+1+3+3) + 0 = 10$$

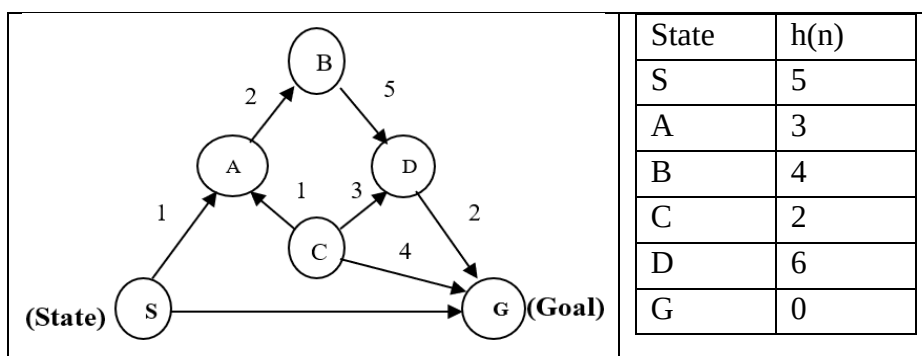
Since $f(J)$ is least, so it decides to go to node J.

➔ Closed list (J)

Shortest Path - $A \rightarrow F \rightarrow G \rightarrow I \rightarrow J$

Path Cost is $3+1+3+3=10$

Example 2: Consider the following graph and apply A* algorithm and find the most cost-effective path to reach from start state **S** to **final** state G. The heuristic function value of each node n is defined in the table given.



Solution:

$$S \rightarrow A = 1+3 = 4$$

$$S \rightarrow G = 10+0 = 10$$

$$S \rightarrow A \rightarrow B = 1+2+4 = 7$$

$$S \rightarrow A \rightarrow C = 1+1+2 = 4$$

$$S \rightarrow A \rightarrow C \rightarrow D = 1+1+3+6 = 11$$

$$S \rightarrow A \rightarrow C \rightarrow G = 1+1+4 = 6$$

$$S \rightarrow A \rightarrow B \rightarrow D = 1+2+5+6 = 14$$

$$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = 1+1+3+2 = 7$$

$$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = 1+2+5+2 = 10$$

3.8.2 Advantages and disadvantages of A* algorithm

Advantages:

- A* search algorithm is the best algorithm among other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

Disadvantages:

- It does not always produce the shortest path as it is mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is the memory requirement, as it keeps all generated nodes in memory, so it is not practical for various large-scale problems.

3.8.3 Admissibility Properties of A* algorithm

A heuristic is called admissible if it always **underestimates**, that is, we always have $h(n) \leq h^*(n)$, where $h^*(n)$ denotes the minimum distance to a goal state from state n . For finite state spaces, A* always terminates.

In other words, **if the heuristic function h always underestimates, then the true cost h^* (that is, heuristic function cost h) is smaller than true cost $h^*(n)$, then A* is guaranteed to find an optimal solution.**

If there is a path from s to a goal state, A* terminates (even when the state space is infinite). Algorithm A* is admissible, that is, if there is a path from s to a goal state, A* terminates by finding an optimal path. If we are given two or more admissible heuristics, we can take their max to get a stronger admissible heuristic.

Admissibility Condition:

By admissible algorithm, we mean that the algorithm is sure to find an optimal solution if one exists. Please note that this is possible only when the evaluation function value never overestimates the distance of the node to the goal. Also note that if the evaluation function value, which is a heuristic one, is exactly

the same as the distance of the node to the goal, then this algorithm will immediately give the solution.

For example, the A* algorithm discussed above is admissible. There are three conditions to be satisfied for A to be admissible.

They are as follows-

1. Each node in the graph has a finite number of successors (or O).
2. All arcs in the graph have costs greater than some positive amount (say C).
3. For each node in the graph, n , $h(n) \leq h'(n)$.

This implies that the heuristic guess of the cost of getting from node n to the goal is never an overestimate. This is known as a **heuristic condition**. Only if these three conditions are satisfied, A* is guaranteed to find an optimal (least) cost path. **Please note that the A* algorithm is admissible for any node n if on such a path, $h'(n)$ is always less than or equal to $h(n)$.** This is possible only when the evaluation function value never overestimates the distance of the node to the goal. Although the admissibility condition requires $h'(n)$ to be a lower bound on $h(n)$, it is expected that the more closely $h'(n)$ approaches $h(n)$, the better the performance of the algorithm.

If $h(n) = h'(n)$, an optimal solution path would be found without over-expanding a node of the path. We assume that one optimal solution exists. If $h'(n)=0$, then A* reduces to the blind uniform cost algorithm or breadth-first algorithm.

Please note that the admissible heuristics are by nature optimistic because they think that the cost of solving the problem is less than it actually is because $g(n)$ is the exact cost for each n . Also note that $f(n)$ should never overestimate the true cost of a solution through n .

For example, consider a network of roads and cities with roads connecting these cities. Our problems to find a path between two cities such that the mileage/fuel cost is minimal. Then an admissible heuristic would be to use distance to estimate the costs from a given city to the goal city. Naturally, the air distance will be either equal to the real distance or will underestimate it, i.e., $h(n) \leq h'(n)$.

3.8.4 Properties of the Heuristic Algorithm:

1. **Admissibility condition:** Algorithm A is admissible if it guarantees to return an optimal solution when one exists. A heuristic function h is called admissible if, in general, we always have $h(n) \leq h'(n)$.*
2. **Completeness condition:** Algorithm A is complete if it always terminates with a solution when one exists.
3. **Dominance property:** If A_1 and A_2 are two admissible versions of Algorithm A such that A is more informed than A_2 , $h_1(n) > h_2(n)$.
4. **Optimal Property:** Algorithm A is optimal over a class of algorithms if it dominates all members of the class.

3.8.5 Results on A* Algorithm

1. **A* is admissible: Algorithm A* is admissible**, that is, if there is a path from S to the goal state, A* terminates by finding an optimal solution.
2. **A* is complete**: If there is a path from S to the goal state, **A terminates** (Even when the state space is ∞).
3. **Dominance property**: If A_1 and A_2 are two admissible versions of A* such that A_1 is more informed than A_2 , then A_2 expands at least as many states as does A_1 . (So A_1 dominates A_2 here, because it is a better heuristic than A_2)

If we are given two or more admissible heuristics, we can take their max to get a stronger admissible heuristic.

3.9 PROBLEM REDUCTION SEARCH

Problem reduction search is broadly defined as planning how best to solve a problem that can be recursively decomposed into subproblems in multiple ways. There are many ways to decompose a problem; we must find the best decomposition, one that provides the highest search efficiency or incurs the minimum cost.

We already know about the divide and conquer strategy; a solution to a problem can be obtained by decomposing it into smaller sub-problems. Each of these sub-problems can then be solved to get its sub-solution. These sub-solutions can then be recombined to get a solution as a whole. That is called **Problem Reduction**. This method generates an arc, which is called an AND arc. One AND arc may point to any number of successor nodes, all of which must be solved for an arc to point to a solution.

When a problem can be divided into a set of sub-problems, where each sub-problem can be solved separately, and a combination of these will be a solution, AND-OR graphs or AND - OR trees are used for representing the solution. The decomposition of the problem or problem reduction generates AND arcs. Consider the following example to understand the AND-OR graph (figure-11).

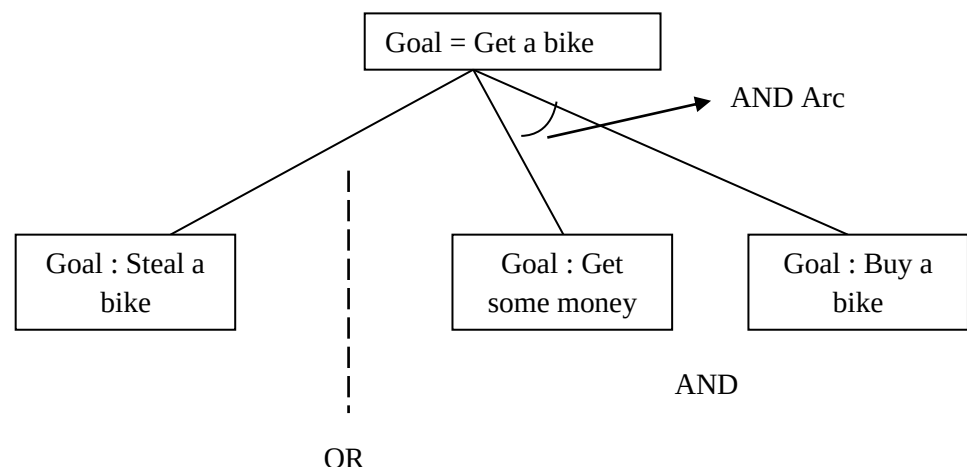


Fig 11: AND-OR graph

Figure 11 shows an AND-OR graph. In an AND-OR graph, an OR node represents a choice between possible decompositions, and an AND node represents a given decomposition. For example, to get a bike, we have two options:

1. (Steal a bike)

OR

2. Get some money **AND** buy a bike.

In this graph, we are given two choices: first, steal a bike or get some money **AND** buy a bike.

When we have more than one choice, and we have to pick one, we apply the OR condition to choose one. That's what we did here.

Basically, the ARC here denotes an AND condition.

Here we have replicated the arc between the get some money and buy a bike, because by getting some money, the possibility of buying a bike is greater than that of stealing one.

AO* search algorithm is based on an AND-OR graph, so it is called AO* search algorithm. AO* Algorithm is basically based on problem decomposition (breaking down the problem into small pieces).

The main difference between the A*(A star) and AO*(AO star) algorithms is that the A* algorithm represents an OR graph algorithm that is used to find a single solution (either this or that). But an **AO* algorithm** represents an AND-OR graph algorithm that is used to find more than one solution by ANDing more than one branch.

A* algorithm guarantees to give **an optimal solution**, while AO* doesn't since AO* doesn't explore all other solutions once it gets a solution.

3.9.1 Problem definition in AND-OR graph:

Given $[G, s, T]$

Where **G**: Implicitly specified AND/OR graph

s: Start node of the AND/OR graph

T: Set of terminal nodes (called SOLVED)

$h(n)$: Heuristic function estimating the cost of solving the sub-problem at n .

Example 1:

Let us see one example with the presence of heuristic value at every node (see Fig 12). The estimated heuristic value is given at each node. The heuristic value $h(n)$ at any node indicates "from this node at least $h(n)$ value (or cost) is required to find a solution". Here, we assume the edge cost value (i.e., $g(n)$ value) for each edge is 1. Remember, in the OR node, we always mark that successor node which indicates the best path for the solution.

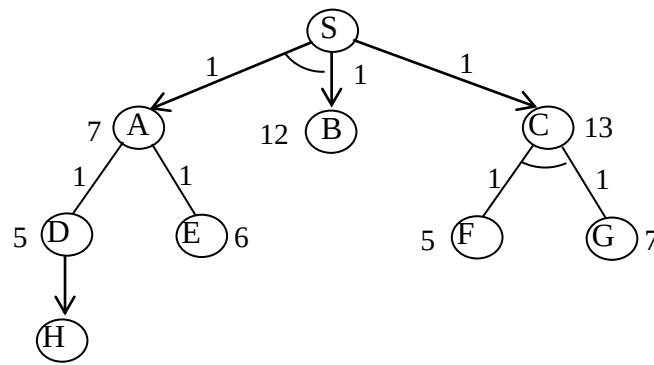


Fig 12: AND-OR graph with heuristic value at each node

Note that, the graph given in Fig 12, there are two paths for the solution from start state S: either S-A-B or S-C. To calculate the cost of the path, we use the formula $f(n) = g(n) + h(n)$ [note that here the $g(n)$ value is 1 for every edge].

Path1: $f(S-A-B) = 1+1+7+12=21$

Path2: $f(S-C) = 1+13=14$

Since $\min(21, 14) = 14$; so, we select successor node C, as its cost is minimum, which indicates the best path for the solution.

Note that C is an AND node; so, we consider both the successor nodes of C. The cost of node C is $f(C-F-G) = 1+1+5+7=14$; so, the revised cost of node C is 14, and now the revised cost of node S is $f(S-C) = 1+14=15$ (revised).

Note that once the cost (that is f value) of any node is revised, we propagate this change backward through the graph to decide the current best path.

Now, let us explore another path and check whether we are getting a lower cost than this cost or not.

$f(A-D)=1+5=6$ and $f(A-E)=1+6=7$; since A is an OR node so best successor node is D since $\min(6,7)=6$. So the revised cost of Node A will be 6 instead of 7, that is $f(A)=6$ (revised). Now next selected node is D, and D has only one node H so $f(D-H)=1+2=3$, so the revised cost of node D is 3, so now, the revised cost of node A is $f(A-D-H)=4$. This path is better than $f(A-E)=7$. So, the final revised cost of node A is 4. Now the final revised cost of $f(S-A-B)=1+1+4+12=18$ (revised).

Thus, the final revised cost for

Path1: $f(S-A-B)=18$ and

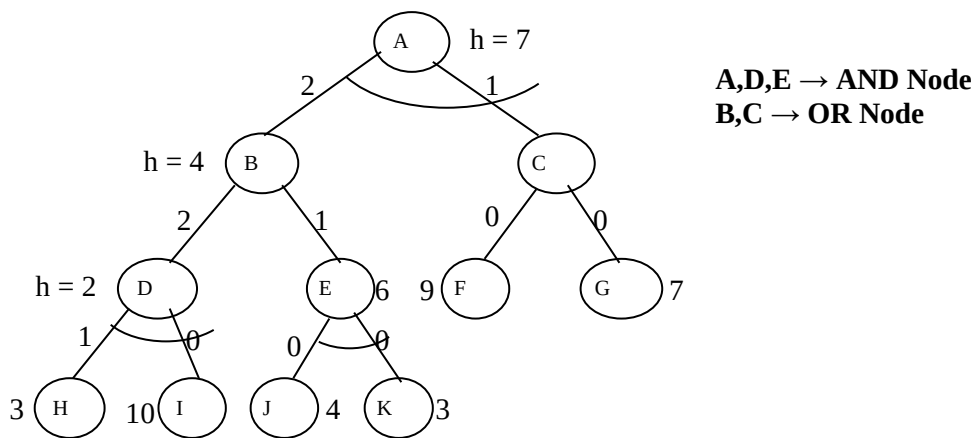
Path2: $f(S-C)=15$

So the optimal cost is 15.

Example 2:

Consider the following AND-OR Graph with estimated heuristic cost at every node. Note that A, D, E are AND nodes and B, C are OR nodes. Edge cost (i.e., $g(n)$ value) is also given.

Apply the AO* algorithm and find the optimal cost path using the AO* algorithm.

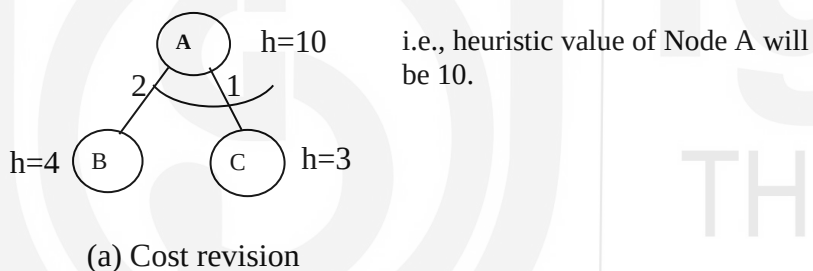


AND-OR graph with heuristic function value.

Heuristic (Estimated) cost at every node is given. For example, heuristic cost at node A is $h=7$, which means at least a 7-unit cost is required to find a solution. Since A is an AND Node, we have to solve both of its successor Nodes, B and C.

Cost of Node A i.e., $f(A-B-C) = (2+4) + (1+3) = 10$

We perform cost revision in a bottom-up fashion.

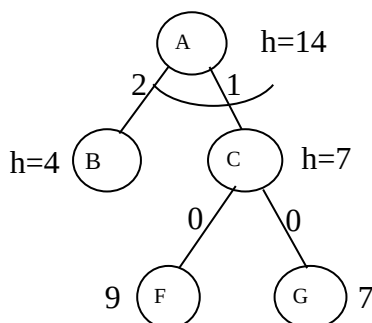


Now, let's see R.H.S first. Node C is an OR Node.

So, cost $f(C-F) = 0+9=9$

& $f(C-G) = 0+7=7$

So, the best successor of C is G, so we perform cost revision in Bottom-up fashion as follows:



Now, the revision cost of node A is 14; So, till now, we can say that the best path to solve the problem is: G-C-A. Now expand the left node of the root, i.e., node B.

Since B is an OR Node; So, we have to see which successor is best, i.e., D or E.

The revised cost of node D is $= (3+1) + (10+0) = 14$

And the revised cost of node E $= (4+0) + (3+0) = 7$

So, the best promising successor node is E.

Now, perform cost revision in a bottom-up fashion.

Note that all the leaf node in the marked tree is solved. So, the best way to solve the problem is to follow the marked tree and solve those marked problems. The best cost to solve the problem is **18**.

Note that for the **AND Node**: if both successor (unit problem) is solved, then we declare SOLVED, and for **the OR Node**: if any one best successor is SOLVED, then we declare SOLVED.

3.9.2 AO* algorithm

Our real-life situations cannot be exactly decomposed into either an AND tree or an OR tree, but are always a combination of both. So, we need an AO* algorithm where O stands for 'ordered'. Instead of two lists OPEN and CLOSED of A* algorithm, we use a single structure GRAPH in AO* algorithm. It represents a part of the search graph that has been explicitly generated so far. Please note that each node in the graph will point both down to its immediate successors and to its immediate predecessors. Also note that each node will have some $h'(n)$ value associated with it. But unlike A* search, $g(n)$ is not stored. It is not possible to compute a single value of $g(n)$ due to many paths to the same state. It is not required, also, as we are traversing top-down along the best-known path.

This guarantees that only those nodes that are on the best path are considered for expansion

Hence, $h'(n)$ will only serve as the estimate of the goodness of a node.

Next, we develop an AO* algorithm.

Algorithm AO*

1. Initialize: Set $G^* = \{s\}$, $f(s) = h(s)$
If $s \in T$, label s as SOLVED
2. Terminate: If s is SOLVED, then Terminate
3. Select: Select a non-terminal leaf node n from the marked sub-graph
4. Expand: Make explicit the successors of n for each new successors, m :
Set $f(m) = h(m)$
If m is terminal, label m SOLVED
5. Cost Revision: Call **Cost-Revise (n)**
6. Loop: Go to Step 2.

Cost Revision in AO*: Cost-Revise(n)

1. Create $Z = \{n\}$
2. If $Z = \{\}$ return
3. Select a node m from Z such that m has no descendants in Z
4. If m is an AND node with successors $r_1, r_2, \dots, r_k$:

Set $f(m) = \sum [f(r_i) + c(m, r_i)]$

Mark the edge to each successor of m

If each successor is labelled SOLVED,
then label m as SOLVED.

5. If m is an OR node with successor $r_1, r_2, \dots, r_k$:

Set $f(m) = \min \{f(r_i) + c(m, r_i)\}$

Mark the edge to the best successor of m

If the marked successor is labelled SOLVED, label m as SOLVED

6. If the cost or label of m has changes, then insert those parents
of m into Z for which m is a marked successor

3.9.3 Advantage of AO* algorithm

Note that **AO*** will always find a minimum cost solution if one exists if $h'(n) < h(n)$ and that all arc costs are positive. The efficiency of this algorithm will depend on how closely $h'(n)$ approximates $h(n)$. Also note that **AO*** is guaranteed to terminate even on graphs that have cycles.

Note: When the graph has only OR node, then the AO* algorithm works just like A* algorithm.

3.10 MEMORY BOUND HEURISTIC SEARCH

The following are the commonly used memory-bound heuristics search:

1. Iterative deepening A* (IDA*)
2. Recursive Best-First search (RBFS)
3. Memory-bound A* (MBA*)

3.10.1: Iterative Deepening A* (IDA*)

IDA* is a variant of the A* search algorithm, which uses iterative deepening to keep the memory usage lower than in A*. It is an informed search based on the idea of the uniformed iterative deepening search. Iterative deepening A* or IDA* is similar to iterative-deepening depth-first, but with the following modifications:

The depth bound is modified to be an f-limit

1. Start with limit = $f(\text{start})$
2. Prune any node if $f(\text{node}) > f\text{-limit}$
3. Next f-limit = minimum cost of any node pruned

Iterative Deepening is a kind of uniformed search strategy. It combines the benefits of depth-first and breadth-first search.

Advantage of IDA* is: It is optimal and complete like breadth-first search, and has a modest memory requirement like depth-first search.

IDA* algorithm

1. Set $C = f(s)$
2. Perform DFBB with cut-off C
Expand a state, n , only if its f -value is less than or equal to C
If a goal is selected for expansion, then return C and terminate
3. Update C to the minimum f -value which exceeded C among states that were examined, and Go TO Step 2.

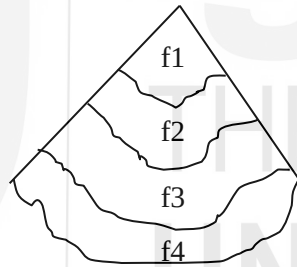
3.10.2 Working of IDA*

- Perform depth-first search LIMITED to some f -bound.
- If the goal is found, then ok.
- Else: increase the f -bound and restart.

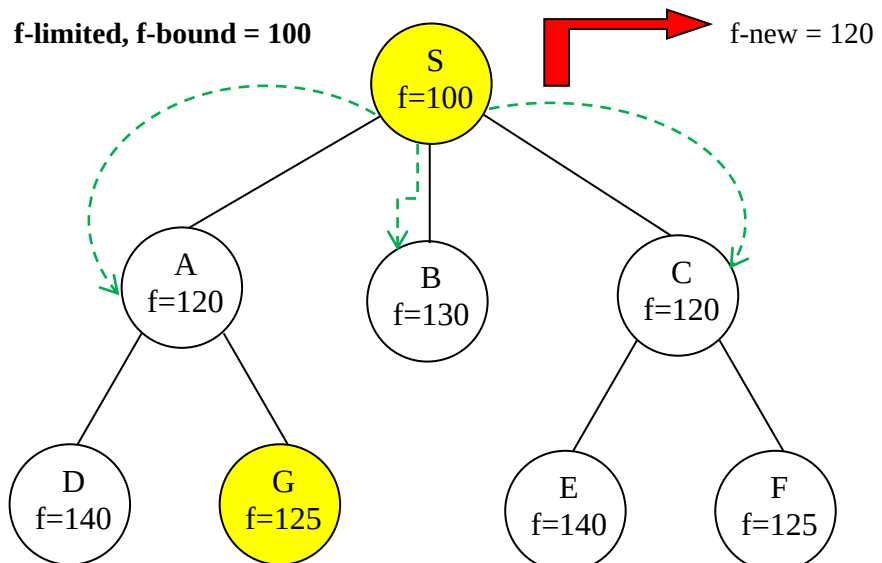
How to establish the f -bound?

Initially: $f(S)$

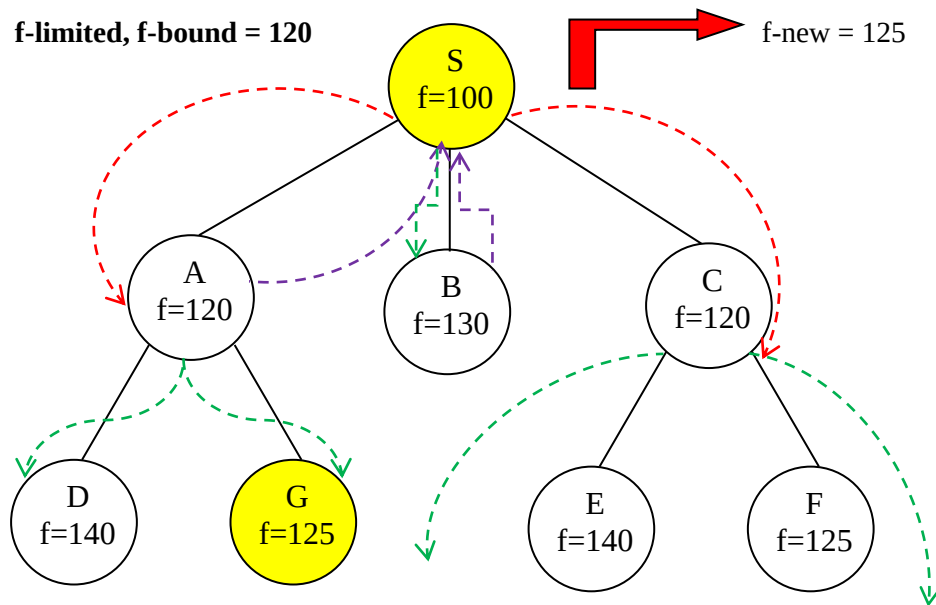
- Generate all successors
- Record the minimal $f(\text{succ}) > f(S)$
- Continue with minimal $f(\text{succ})$ instead of $f(S)$



Consider the following example to understand the IDA*

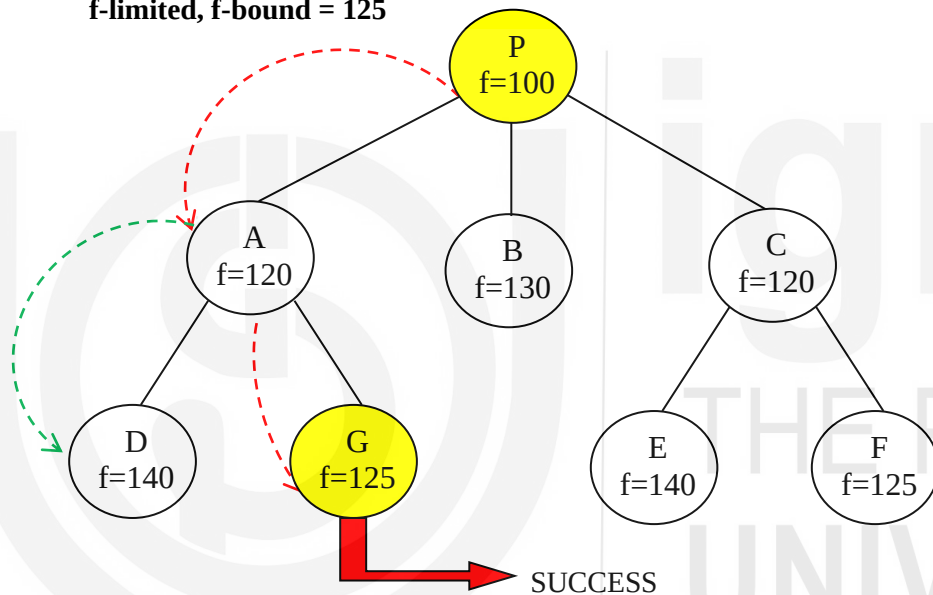


f-limited, f-bound = 120



Uninformed & Informed Search

f-limited, f-bound = 125



3.10.3 Analysis of IDA*

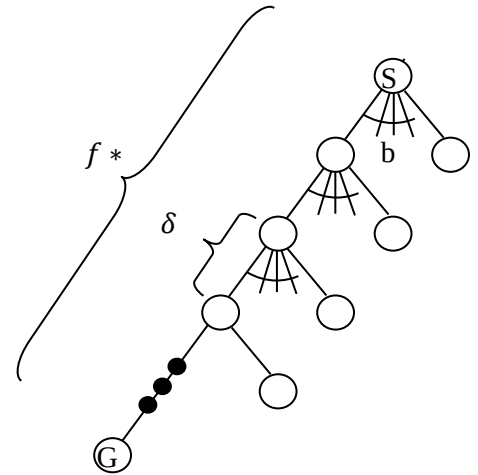
IDA* is complete, optimal, and optimally efficient (assuming a consistent, admissible heuristic), and requires only a polynomial amount of storage in the worst case:

F^* = optimal path cost to a goal

B = branching factor

δ = minimum operator step cost

$\frac{bf^*}{\delta}$ nodes of storage required



Note that IDA* is complete and optimal. Space usage is linear in the depth of the solution. Each iteration is a depth-first search, and thus it does not require a priority queue.

3.10.4 Comparison of A* and IDA* algorithm:

- Iterative Deepening Search (IDS) is nothing but BFS plus DFS for tree search.
- IDA* algorithm is “complete and optimal” algorithm.
- BFA and A* are good for optimality, but not memory.
- DFS: good for memory $O(bd)$, but not optimality
- In the worst case, only one new state is expanded in each iteration
- (IDA*). If A* expands N states, then IDA* can expand:

$$1+2+3+\dots+N=O(N^2)$$

3.11 RECURSIVE BEST FIRST SEARCH (RBFS)

The idea of recursive best first search is to simulate **A*** search with **$O(bd)$** memory, where b is the branching factor, and d is the solution depth.

It is a memory-bound, simple recursive algorithm that works like a standard best first search but only takes up linear space. There are some things that make it different from recursive DFS. It keeps track of f , the value of the best alternative path that can be found from any ancestor of the current node, instead of continuing indefinitely down the current path.

RBFS mimics the operation of the standard Best-First search algorithm. RBFS keeps track of the f -value of the best alternative path available from any ancestor of the current node. If the current node exceeds the limit, the recursion unwinds back to the alternative path. As the recursion unwinds, RBFS replaces the f -value of each node along the path with the best f -value of its children. In this way, RBFS remembers the f -value of the best leaf in the forgotten subtree and can therefore decide whether it's worth re-expanding the subtree at some later time.

RBFS is somewhat more efficient than IDA*, but still suffers from excessive node regeneration. A* and RBFS are optimal algorithms if the heuristic function $h(n)$ is admissible.

Uninformed & Informed Search

```

function RECURSIVE-BEST-FIRST-SEARCH(problem) returns a solution, or failure
  return RBFS(problem, MAKE-NODE(problem.INITIAL-STATE),  $\infty$ )

function RBFS(problem, node, f-limit) returns a solution, or failure and a new f-cost limit
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  successors  $\leftarrow$  []
  for each action in problem.ACTIONS(node.STATE) do
    add CHILD-NODE(problem, node, action) into successors
  if successors is empty then return failure,  $\infty$ 
  for each s in successors do /* update f with value from previous search, if any */
    s.f  $\leftarrow$  max(s.g + s.h, node.f)
  loop do
    best  $\leftarrow$  the lowest f-value node in successors
    if best.f > f-limit then return failure, best.f
    alternative  $\leftarrow$  the second-lowest f-value among successors
    result, best.f  $\leftarrow$  RBFS(problem, best, min(f-limit, alternative))
    if result  $\neq$  failure then return result

```

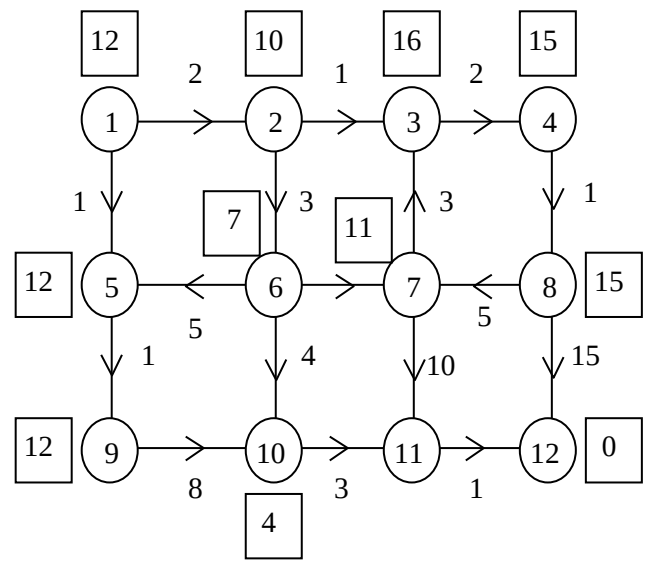
Algorithm for recursive Best First Search

3.11.1 Advantages and Disadvantages of RBFS:

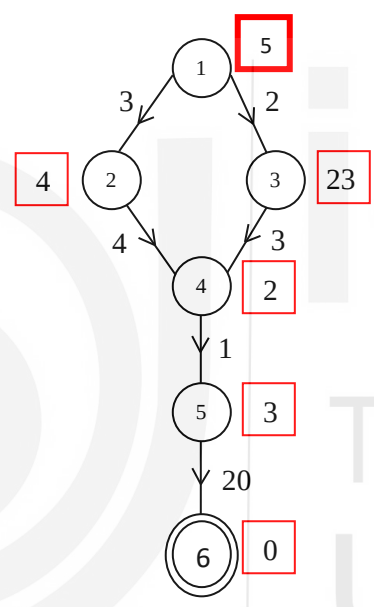
- More efficient than IDA* and still optimal.
- Best-first Search based on next best f -contour; fewer regenerations of nodes.
- Exploit the results of the search at a specific f -contour by saving the next f -contour associated with a node whose successors have been explored.
- Like IDA*, RBFS still suffers from excessive node regeneration.
- IDA* and RBFS are not good for graphs.
- Can't check for repeated states other than those on the current path.
- Both are hard to characterize in terms of expected time complexity.

☞ Check Your Progress 2

Q.1 Apply A* on the following graph: -

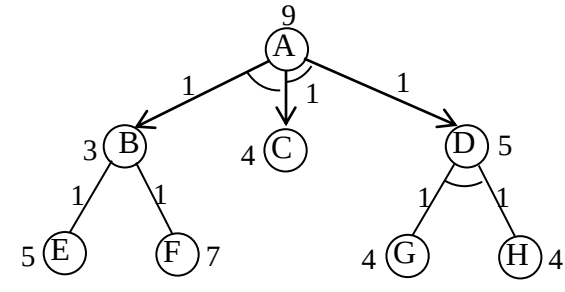


Q.2 Apply A* algorithm on the following graph

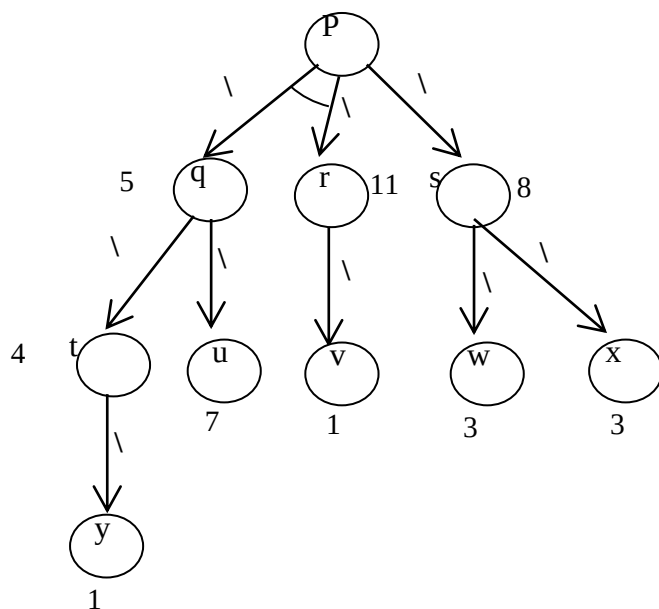


Q.3 Differentiate between the A* and AO* algorithms.

Q.4: Apply AO* algorithm on the following graph. Heuristic value is also given at every node, and assume the edge cost value of each node is 1.



Q.5 Apply AO* algorithm on the following graph. Heuristic value is also given at every node and assume the edge cost value of each node is 1.



Example 6: Given the 3 matrices A1, A2, A3 with their dimensions (3×4) , (4×10) , (10×1) , respectively. Consider the problem of solving this **chain matrix multiplication**. Apply the concept of AND-OR graph and find a minimum cost solution tree.

(Multiple choice Questions)

Q.6 A* algorithm always finds an optimal solution if

- A. h' is always 0
- B. g is always 1
- C. h' never overestimates h
- D. h' never underestimate h

Q.7 A* algorithm uses $f^* = g + h^*$ to estimate the cost of getting from the initial state to the goal state, where g is a measure of the cost of getting from the initial state to the current node, and the function h^* is an estimate of the cost of getting from the current node to the goal state. To find a path involving the fewest number of steps, we should test,

- A. $g=1$
- B. $g=0$
- C. $h^* = 0$
- D. $h^* = 1$

3.12 SUMMARY

- As the name ‘Uninformed Search’ means the machine blindly follows the algorithm regardless of whether right or wrong, efficient or inefficient.
- These algorithms are brute force operations, and they don’t have extra information about the search space; the only information they have is on how to traverse or visit the nodes in the tree. Thus, uninformed search algorithms are also called **blind search** algorithms.
- The search algorithm produces the search tree without using any domain knowledge, which is a brute force in nature. They don’t have

any background information on how to approach the goal or whatsoever. But these are the basics of search algorithms in AI.

- **The different types of uninformed search algorithms are as follows:**
 - Depth First Search
 - Breadth-First Search
 - Depth Limited Search
 - Uniform Cost Search
 - Iterative Deepening Depth First Search
 - Bidirectional Search (if applicable)
- The following terms are frequently used in any search algorithms:
 - **State:** It provides all the information about the environment.
 - **Goal State:** The desired resulting condition in a given problem and the kind of search algorithm we are looking for.
 - **Goal Test:** The test to determine whether a particular state is a goal state.
 - **Path/Step Cost:** These are integers that represent the cost to move from one node to another node.
- To evaluate and compare the efficiency of any search algorithm, the following 4 properties are used:
 - **Completeness:** A search algorithm is said to be complete if it guarantees to return a solution if exist.
 - **Optimality/Admissibility:** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution.
 - **Space Complexity:** A function describing the amount of space(memory) an algorithm takes in terms of input to the algorithm. That is how much space is used by the algorithm? Usually measured in terms of the maximum number of nodes in memory at a time.
 - **Time Complexity:** A function describing the amount of time the algorithm takes in terms of input to the algorithm. That is, how long (worst or average case) does it take to find a solution?
- Time and space complexity are measured in terms of: ‘b’ – maximum branching factor (the maximum number of successors (children) of any node) in a tree, ‘d’ – the depth of the shallowest goal node, and ‘m’ – maximum depth of the search tree (maybe infinity).
- The following table summarizes the 4 properties (**Completeness, Optimality/Admissibility, Space Complexity, Time Complexity**) of the search algorithm.

	BFS	DFS	IDDFS	Bidirectional Search (if Applicable)
Time	bd	bd	bd	bd/2
Space	bd	bm	bd	bd/2
Optimum?	Yes	No	Yes	Yes
Complete?	Yes	No	Yes	Yes

- The advantage of DFS is that it requires very little memory as compared to BFS, as it only needs to store a stack of the nodes on the path from the root node to the current node. The disadvantages of DFS are as follows: There is the possibility that many states keep recurring,

and there is no guarantee of finding the solution. The DFS algorithm goes for deep down searching, and sometimes it may go into an infinite loop.

- On the other hand, BFS gives a guarantee to get an optimal solution, if any solution exists (Completeness) and if there is more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps (Optimal), but one major drawback of BFS is that it requires lots of memory space since each level of the tree must be saved into memory to expand the next level.
- Iterative Deepening DFS (IDDFS) combines the benefits of both BFS and DFS search algorithms in terms of fast search and memory efficiency. It is better than DFS and needs less space than BFS. But the main drawback of IDDFS is that it repeats all the work from the previous phase.
- The advantage of Bidirectional search (BS) is that it uses various techniques like DFS, BFS, DLS, etc, so it is efficient and requires less memory. The Implementation of the bidirectional search tree is difficult, and in bidirectional search, one should know the goal state in advance.
- In Informed search, the domain-dependent (heuristic) information is used in order to search the space more efficiently.
- Informed searches include the following search:
 - **Best-first search:** Order agenda based on some measure of how 'good' each state is.
 - **Uniform-cost search:** Cost of getting to current state from initial state = $g(n)$
 - **Greedy search:** Estimated cost of reaching goal from current state – Heuristic evaluation functions, $h(n)$
 - **A* search:** $f(n) = g(n) + h(n)$
- Admissibility: $h(n)$ never overestimates the actual cost of getting to the goal state.
- Informed Ness: A search strategy that searches fewer of the states person orders to find a goal state is more informed.
- A* algorithm avoids expanding paths that are already expensive.
- In A*, the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ = cost so far to reach n , $h(n)$ = estimated cost to goal from n , and $f(n)$ = estimated total cost of path through n to goal.
- A* search uses an admissible heuristic function, i.e., $h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost of the cheapest solution from n .
- A* search has a very good property: A* search is optimal! So, if there is any solution, A* search is guaranteed to find a least cost solution. Remember, this needs an admissible heuristic.
- The commonly used memory bound heuristics search are Iterative deepening A* (IDA*), Recursive Best-First search (RBFS), and Memory-bound A* (MBA*).
- Iterative Deepening Search (IDS) is nothing but BFS plus DFS for tree search. IDA* algorithm is "complete and Optimal" algorithm.
- The idea of recursive best first search is to simulate A* search with **O(bd)** memory, where b is the branching factor, and d is the solution depth.

3.13 SOLUTIONS/ANSWERS

Check your progress 1:

Answer 1:

Table 1: Open and closed list for BFS

OPEN LIST	CLOSED LIST
1	1
2,3,4	2
3,4,6	3
4,6,5	4
6,5,7	5
5,7	6
7,	7
8-Goal State	-

Table 2: Open and closed list for DFS

OPEN LIST	CLOSED LIST
1	1
4,3,2	2
4,3,6	6
4,3	3
4,5	5
4,7	7
4,8-Goal State	-

Answer 2

- open = [A]; closed = []**
- open = [B,C,D]; closed = [A]** B is not the goal.
Put his children onto the queue.
- open = [C,D,E,F]; closed = [B,A]** Put him in closed. He is done.
- open = [D,E,F,G,H]; closed =[C,B,A]** Items between red bard are siblings.
- open = [E,F,G,H,I,J]; closed = [D,C,B,A]**
- open = [F,G,H,I,J,K,L]; closed = [E,D,C,B,A]**
- open = [G,H,I,J,K,L,M,N]** (as L is already on open);**closed = [G,F,E,D,C,B,A]**
- open = [H,I,J,K,L,M,N]; closed = [G,F,E,D,C,B,A]**
- and so on until either goal is reached or open is empty.

Answer 3: Refer BFS section for solution.

Answer 4

	BFS	DFS	IDDFS	Bidirectional Search (if Applicable)

Time	b^d	b^d	b^d	$b^{d/2}$
Space	b^d	bm	bd	$b^{d/2}$
Optimum?	Yes	No	Yes	Yes
Complete?	Yes	No	Yes	Yes

Uninformed & Informed
Search

Check your progress 2

Answer 1:

The OPEN and CLOSED list are shown below. Node with their $f(n)$ values are inserted in OPEN list and that node will be expended next whose $f(n)$ value is minimum.

CLOSED

$1^{(12)}$	$2^{(12)}$	$6^{(12)}$	$5^{(13)}$	$10^{(13)}$	$11^{(13)}$	$12^{(13)}$
------------	------------	------------	------------	-------------	-------------	-------------

OPEN

$1^{(12)}$			
$2^{(12)}$	$5^{(13)}$		
$5^{(13)}$	$3^{(14)}$	$6^{(12)}$	
$5^{(13)}$	$3^{(14)}$	$7^{(17)}$	$10^{(13)}$
$3^{(19)}$	$7^{(17)}$	$10^{(13)}$	
$3^{(19)}$	$7^{(17)}$	$10^{(13)}$	$9^{(14)}$
$3^{(19)}$	$7^{(17)}$	$9^{(14)}$	$11^{(13)}$
$3^{(19)}$	$7^{(17)}$	$9^{(14)}$	$12^{(13)}$

Note that only 6 nodes are expended to reach to a goal node. Optimal cost to reach from start state (1) to goal node (12) is **13**.

Note: If all the edge cost is positive then **Uniform cost search** (UCS) algorithm is same as Dijkstra's algorithm. Dijkstra algorithm fails if graph is having a negative weight cycle. A* algorithm allows negative weight also. It means A* algorithm work for negative (-ve) edge cost also. If some edge cost is negative, then at any point of successive iteration, we cannot say till that node we have optimum cost (because of the negative cost). So, in this case (-ve edge cost), nodes come back from CLOSED to OPEN.

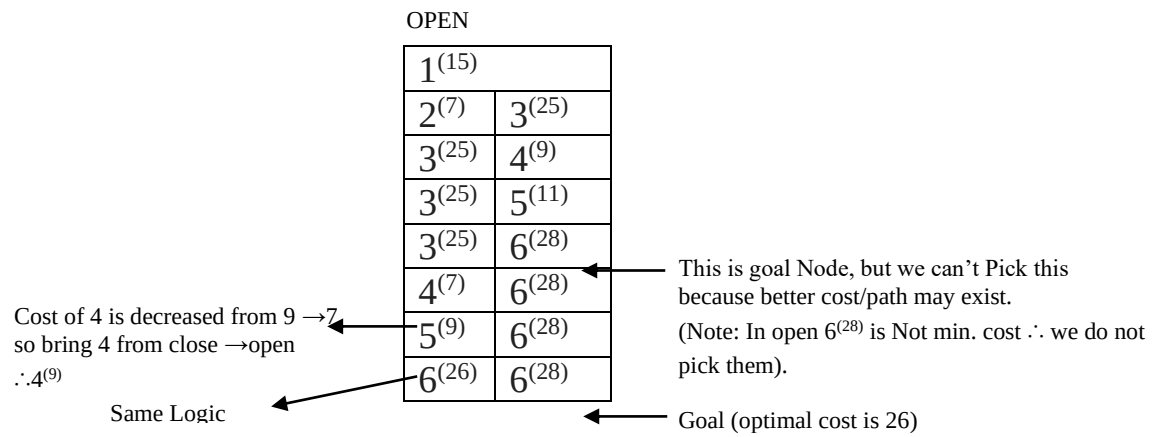
Let us see one example (Example 2) having negative edge cost and you can also see how nodes come back from CLOSED to OPEN.

Answer 2:

The OPEN and CLOSED list are shown below. Node with their $f(n)$ values are inserted in OPEN list and that node will be expended next whose $f(n)$ value is minimum.

CLOSED

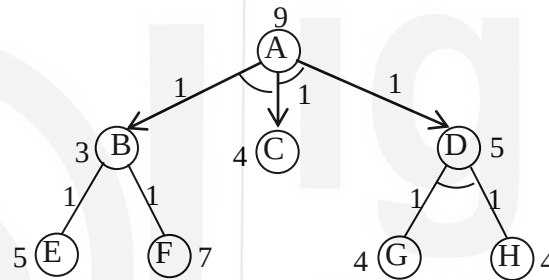
$1^{(15)}$	$2^{(7)}$	$4^{(9)}$	$5^{(11)}$	$3^{(25)}$	$4^{(7)}$	$5^{(9)}$
------------	-----------	---------------------------------	----------------------------------	------------	-----------	-----------



Optimal cost to reach from start state (1) to goal node (6) is **26**.

Answer 3: An A* algorithm represents an OR graph algorithm that is used to find a single solution (either this or that), but an AO* algorithm represents an AND-OR graph algorithm that is used to find more than one solution by ANDing more than one branch.

Answer 4 :



Path - 1: $f(A-B-C) = 1+1+3+4=9$

$f(B-E) = 1+5=6$

$f(B-F) = 1+7=8$

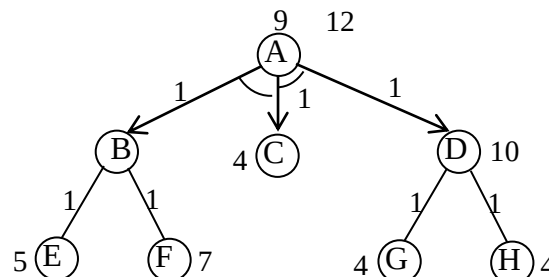
$f(A-B-C) = 1+1+6+4=12$

Path-2 : $f(A-C-D) = 1+1+4+5=11$

$f(D-G-H) = 1+1+4+4=10$

$f(A-C-D) = 1+1+4+10=16$

AND-OR graph with revised cost is shown in figure.



So, the optimal cost $f(A-B-C) = 12$

Note that AO* algorithm does not explore all the solution path once it finds a solution.

Answer 5: Similar to Q.4

Answer 6:

Given a chain of matrices: $A_1, A_2, \dots, A_n$, where each matrix A_i has a dimension $p_{i-1} \times p_i$. Problem is to determine the order of multiplication or the way of parenthesizing the product of matrices, so that the minimum number of required operations (i.e., multiplications) can be minimized.

There are only 2 ways to parenthesizing the given 3 matrices, A_1, A_2, A_3 :

$$A_1 \times (A_2 \times A_3)$$

or

$$(A_1 \times A_2) \times A_3$$

As we know, if matrix A is of dimension $(p \times q)$ and matrix B is of dimension $(q \times r)$, then the cost of multiplying A to B that is $(A \times B)$ is $(p \times q \times r)$ and the final dimension of the resultant matrix $(A \times B)$ is $(p \times r)$.

Let us see how AND-OR graph is used to get the solution of this problem.

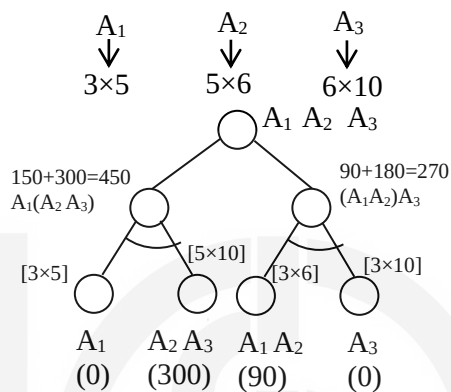


Figure-1 AND/OR graph for multiplying $A_1 \times A_2 \times A_3$

In this AND-OR graph, parent (root) node indicates the given problem for multiplying $A_1 A_2 A_3$. Next level of the tree (2 successors node) indicates the 2 choices (or ways) of multiplying (or parenthesizing) the $A_1 A_2 A_3$; first way is $A_1 \times (A_2 \times A_3)$ and another way is $(A_1 \times A_2) \times A_3$. Since out of these two choices, anyone will be the solution so there is a OR node for this. In an OR node, we always mark the current best successor node. Next level we have an AND node. For any AND node we must add the cost of both the successor node.

Cost of multiplying $(A_2 \times A_3) = 5 \times 6 \times 10 = 300$ and dimension of $A_2 \times A_3$ is (5×10) . Since the dimension of A_1 is (3×5) and the dimension of $A_2 \times A_3$ is (5×10) , so the cost of multiplying $A_1 \times (A_2 \times A_3) = 3 \times 5 \times 10 = 150$. Thus the total cost will be $300+150=450$. Similarly,

The cost of multiplying $(A_1 \times A_2) = 3 \times 5 \times 6 = 90$ and dimension of $A_1 \times A_2$ will be (3×6) . Since the dimension of $A_1 \times A_2$ is (3×6) and the dimension of A_3 is (6×10) , so the cost of multiplying $(A_1 \times A_2) \times A_3 = 3 \times 6 \times 10 = 180$. Thus the total cost will be $90+180=270$. So, the best way to multiplying $A_1 \times A_2 \times A_3$ is $(A_1 \times A_2) \times A_3$ and the minimum cost of multiplying $A_1 \times A_2 \times A_3$ is 270.

Multiple Choice Questions

Answer 6: Option C

3.14 FURTHER READINGS

1. Ela Kumar, “Artificial Intelligence”, IK International Publications
2. E. Rich and K. Knight, “Artificial intelligence”, Tata Mc Graw Hill Publications
3. N.J. Nilsson, “Principles of AI”, Narosa Publ. House Publications
4. John J. Craig, “Introduction to Robotics”, Addison Wesley publication
5. D.W. Patterson, “Introduction to AI and Expert Systems" Pearson publication

